

Do sistema inteiro à linha de código: tudo que a P1 cobra, em ordem.

Este guia foi montado a partir dos slides das aulas 1 a 5, do roteiro da aula 6, das 7 listas de exercícios, do projeto `designpatterns` (os 23 padrões em Java), das 3 soluções do `crud` e das suas anotações das aulas 1 a 4. O fio condutor é um **zoom**: o curso começa olhando o sistema de longe (estilos de arquitetura), desce para as classes (padrões GoF), explica *por que* esses desenhos são bons (GRASP e SOLID), mostra quem manda no fluxo (IoC, DI, MVC) e termina na camada de dados (DAO, VO, ORM, JPA).

Em paralelo corre uma segunda história: a aplicação **crud de produto**. A cada aula ela perde um acoplamento: primeiro ganha um Singleton, depois DAO e VO, por fim JPA atrás de uma Abstract Factory. Se você entender essa evolução, entende a matéria.

10	23	9 + 5	1	50%
módulos de conteúdo	padrões GoF em 3 grupos	princípios GRASP e SOLID	aplicação-âncora: o crud	da nota · não há P2

Das suas anotações: "IMPORTANTE ESTUDAR ESSES PATTERNS. CAI NA PROVA" (aula 2) · "Aula importante pra prova, conteúdo pesado" (aula 4: GRASP, SOLID, IoC, DI, MVC) · "Sempre fazer exercícios da aula" (aula 1). Esses três avisos definem as prioridades deste guia.

Sumário

Mapa mental geral2

Ordem de estudo3

MÓDULOS

1 · Arquitetura: conceitos4

2 · Estilos arquiteturais7

3 · GoF: padrões de criação9

4 · GoF: padrões estruturais12

5 · GoF: padrões comportamentais15

6 · GRASP19

7 · SOLID22

8 · Hollywood, IoC, DI e MVC24

9 · Persistência: DAO, VO e ORM27

10 · JPA + Abstract Factory no crud31

FERRAMENTAS

Java: colinha de código35

Listas que caem (decorar)39

Auto-teste (flashcards)41

Lacunas e o que NÃO cai45

■ Por que estudar isso

■ Provável na prova

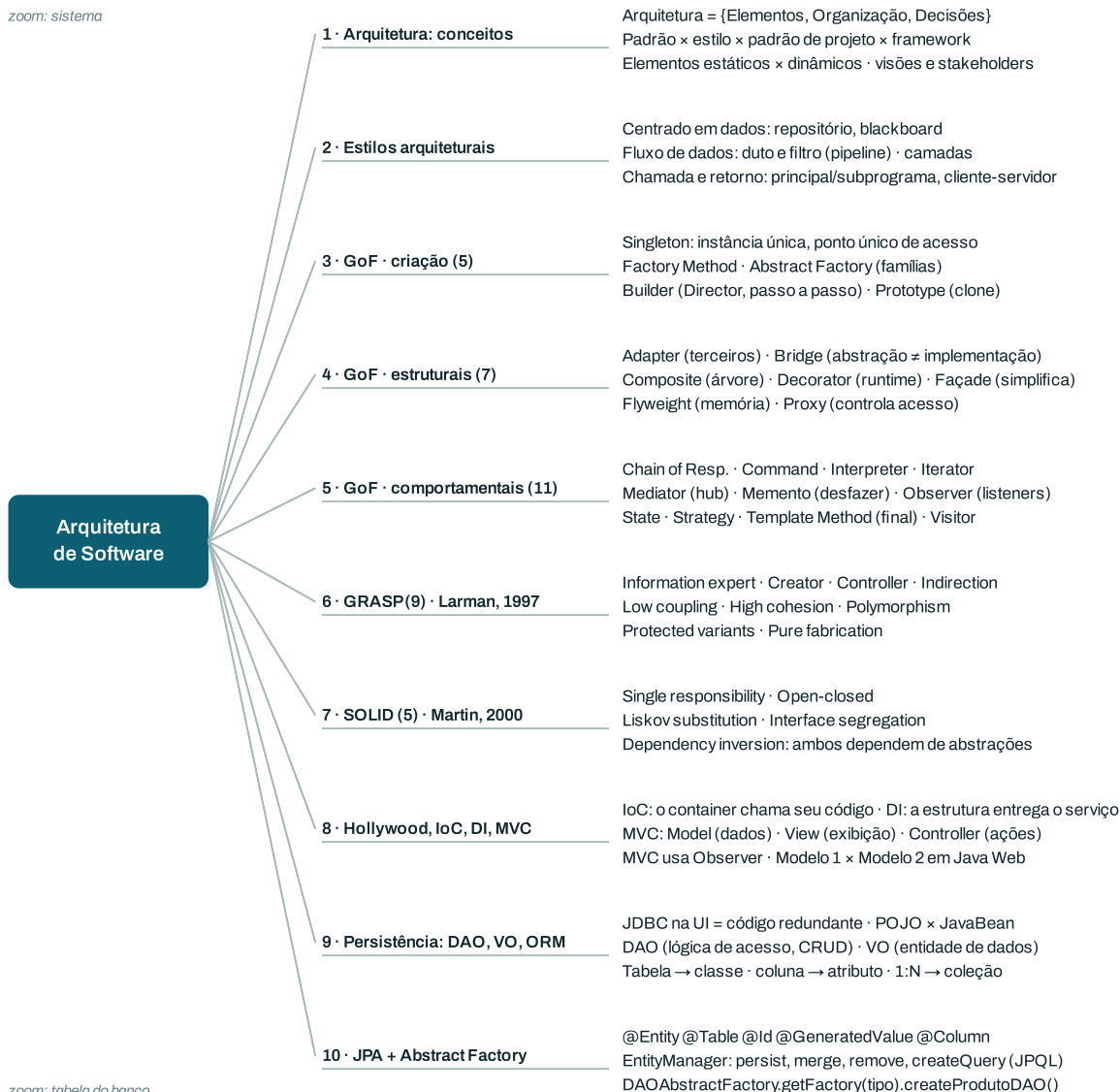
■ Código Java

INFERÊNCIA = não está no material

Mapa mental geral

Leia de cima para baixo: é o zoom do curso, do sistema inteiro até a tabela do banco. Os números entre parênteses são contagens que o professor pode cobrar.

zoom: sistema



zoom: tabela do banco

Ordem de estudo recomendada

A ordem segue as dependências: GRASP cita padrões GoF (Indirection = Adapter, Façade, Observer), MVC usa Observer, e o crud final usa Singleton + Abstract Factory + DAO. O tempo sugerido soma cerca de 9 horas.

1 Vocabulário de arquitetura (1 h)

Módulos 1 e 2. Saiba diferenciar padrão, estilo, padrão de projeto e framework, e associar cenário → estilo. Resolva os 2 exercícios da aula 1 (respondidos no módulo 2).

2 Os 23 padrões GoF (3 h)

Módulos 3, 4 e 5. Primeiro decore a classificação (5 / 7 / 11), depois a intenção de cada um em uma frase, depois os pares que confundem. Sua anotação diz que cai: é o maior bloco.

3 GRASP, SOLID, IoC, DI e MVC (2 h 30)

Módulos 6, 7 e 8. "Conteúdo pesado pra prova". As 4 questões da lista da aula 4 são dissertativas e estão com resposta-modelo aqui. Treine escrevê-las à mão.

4 Persistência e o crud (1 h 30)

Módulos 9 e 10. Sem DAO/VO você não explica por que o crud foi reorganizado; sem a Abstract Factory você não fecha a história da aula 6.

5 Escrever de memória + flashcards (1 h)

Feche o guia e escreva: Singleton completo, VendaDeProduto com DI, interface ProdutoDAO, a fábrica abstrata. Depois rode os 36 flashcards.

Plano curto, se só sobrare 3 horas

1) Respostas-modelo da aula 4 (módulos 6 a 8), 60 min. 2) Tabelas dos 23 padrões + exercício 1 e 2 da aula 3 (módulo 5), 60 min. 3) Exercícios da aula 1 (módulo 2), 20 min. 4) Singleton e DI escritos de memória, 20 min. 5) Seção "Listas que caem", 20 min.

Módulo 1 · O que é arquitetura de software

Por que estudar

É o vocabulário da disciplina. Toda questão conceitual parte daqui: "defina arquitetura", "diferencie padrão de framework", "o que são elementos estáticos". São pontos fáceis se você usar os termos exatos dos slides.

As definições de arquitetura (aula 1)

Quem / quando	Definição (palavras do slide)	Palavra-chave
Dijkstra, anos 60	O conceito inicial surgiu aqui (sua anotação: "nome do inventor").	origem
Perry e Wolf (92)	Arquitetura = {Elementos, Organização, Decisões} . Conjunto de elementos arquiteturais (de dados, de processamento, de conexão) que possuem alguma organização; elementos e organização são definidos por decisões tomadas para satisfazer objetivos e restrições.	a tripla
Shaw e Garlan (96)	Define o que é o sistema em termos de componentes computacionais , os relacionamentos entre eles, os padrões que guiam a sua composição e restrições . Vai além de algoritmos e estruturas de dados: controle, protocolos, sincronização, acesso a dados, distribuição física, escalabilidade, desempenho.	componentes + conectores
Bass (98, 2ª ed. 2003)	A estrutura (ou estruturas) do sistema, composta de elementos de software, das propriedades externamente visíveis desses elementos e dos relacionamentos entre eles; é a abstração do sistema .	abstração
IEEE 1471:2000 → ISO/IEC/IEEE 42010:2011	A organização fundamental de um sistema incorporada em seus componentes, seus relacionamentos com o ambiente, e os princípios que conduzem seu design e evolução .	organização fundamental

O que todas têm em comum: **partes + relações entre as partes + as decisões/princípios que justificam esse desenho**. Se der branco na prova, escreva isso e cite Perry e Wolf.

Padrão, padrão de arquitetura, estilo, padrão de projeto e framework

Padrão: template (receita) de solução para um **problema recorrente** que seja **comprovadamente útil** em um determinado **contexto**. É instanciado pela vinculação de valores a seus parâmetros e existe em várias escalas: padrões de arquitetura, de análise, de projeto, de teste e idiomas, de implementação.

Termo	O que é	Escala	Exemplo
Padrão de arquitetura	Template pronto para problema arquitetural recorrente. Expressa um esquema fundamental de organização estrutural; fornece subsistemas predefinidos, suas responsabilidades e regras para as relações entre eles.	sistema	Repositório, duto e filtro, camadas
Estilo de arquitetura	Delimita um conjunto de templates que podem ser escolhidos e impõe um grau de uniformidade . Definido por um conjunto de padrões ou pela escolha de componentes/conectores ("tijolos básicos"). Cada estilo lida com diferentes atributos de qualidade.	sistema	Cliente-servidor, em camadas, orientado a serviços
Padrão de projeto (GoF)	Descreve um problema do nosso ambiente e o cerne da solução, para usar "mais de um milhão de vezes, sem nunca fazê-lo da mesma maneira" (Alexander, 1977, apud Gamma).	classes e objetos	Singleton, Observer
Framework	Denominador comum para uma família de aplicações : coleção de artefatos (em geral classes + software para usá-las) utilizável por várias aplicações. Surge da necessidade de refazer passos semelhantes.	código pronto	Spring, Hibernate, Swing

Os 4 elementos essenciais de um padrão GoF: **nome** (poucas palavras que o identificam), **problema** (contexto, quando aplicar), **solução** (elementos e seus relacionamentos) e **consequências** (resultados, vantagens e desvantagens).

Frameworks: como são usados e de que tipo são

As classes (abstratas ou concretas) de um framework são usadas por **herança, agregação ou dependência**. Relação com arquitetura: a adoção de uma arquitetura (estilo) **permite a escolha de frameworks** que simplifiquem aspectos da aplicação; as classes do framework se integram às classes de domínio por meio de **classes de projeto**, projetadas como extensão imposta pelo framework.

Forma de utilização	Descrição	Exemplo do slide
Conjunto de ferramentas	Coleção de classes úteis, cada uma utilizável isoladamente. Às vezes é chamado de API, não de framework.	—
Denominador comum de arquiteturas	Uma arquitetura abstrata: formas predefinidas de se criar uma aplicação.	AWT/Swing
Sistema que possui controle	"Siga os protocolos para obter sua aplicação": forma rígida de construção; modelos, interfaces e controle bem definidos.	Struts, Spring

Tipo	Característica	Exemplo
Horizontal	Mais generalista, serve a diversas aplicações, maior flexibilidade; realiza funcionalidades que não são as de negócio .	Toolkits gráficos
Vertical	Usado por aplicações de domínio específico ; o contexto do framework está alinhado à finalidade de negócio.	Aplicações estatísticas ou financeiras

Decompondo a definição: elementos e decisões

Aspecto	O que define	Inclui
Elementos estáticos	As partes do sistema e sua organização, e como se relacionam (associações, composições).	Software : módulos, classes, pacotes, procedimentos, serviços. Dados : entidades e tabelas, arquivos, classes de dados. Hardware : computadores, roteadores, cabos, impressoras.
Elementos dinâmicos	O comportamento do sistema: reflete o sistema durante a execução e como reage a estímulos internos e externos.	Processos, módulos, protocolos ou classes que realizam comportamento.
Decisões arquiteturais	As estruturas não são definidas aleatoriamente. O arquiteto escolhe entre alternativas de design, particionando o sistema em elementos e relações que atendam aos atributos de qualidade .	—

Descrição de arquitetura: conjunto de produtos para documentar uma arquitetura. Diferentes interessados (*stakeholders*) veem a descrição sob diferentes **visões**. Visão = "a representação de um sistema como um todo segundo a perspectiva de um conjunto de necessidades (dos interessados) relacionado". A IEEE-Std-1471-2000 estabelece o vocabulário, dá diretrizes para representar a descrição e encoraja práticas consistentes.

Exemplo resolvido: a tripla de Perry e Wolf aplicada ao crud

INFERÊNCIA

Elementos estáticos: de software, as classes `CrudProduto` (janela), `ProdutoDAO`, `DBConnection`; de dados, a tabela `produto` e o VO `Produto`; de hardware, a máquina do MySQL na porta 3306.

Elementos dinâmicos: o que acontece quando o usuário clica em "salvar": a janela monta o VO → pede um DAO à fábrica → o DAO executa o SQL → o banco responde.

Decisão arquitetural: "a janela não fala SQL; todo acesso passa por uma interface DAO". Objetivo atendido: poder trocar JDBC por JPA sem reescrever a janela (manutenibilidade).

Perguntas que o leitor faria

Qual a diferença prática entre padrão de arquitetura e estilo?

O padrão é *uma* receita (ex.: repositório). O estilo é a *família* que delimita quais receitas e quais tijolos (componentes e conectores) você pode usar, impondo uniformidade. Os slides tratam "arquiteturas centradas em dados" como a família e "repositório" e "blackboard" como estilos/padrões dentro dela.

Framework é um padrão?

Não. Padrão é uma ideia (template); framework é **código** reutilizável. Um framework costuma *implementar* vários padrões (Spring implementa IoC/DI, MVC, Singleton nos beans) e é escolhido *depois* que a arquitetura foi adotada.

Swing é horizontal ou vertical?

Horizontal: é um toolkit gráfico, serve a qualquer domínio e não implementa regra de negócio. Um framework de cálculo atuarial seria vertical.

Por que Spring aparece como "sistema que possui controle"?

Porque é ele quem chama o seu código, e não o contrário. Essa é exatamente a Inversão de Controle do módulo 8.

Banco de dados é elemento de software ou de dados?

As **tabelas e entidades** são elementos de dados; o **servidor** onde o SGBD roda é elemento de hardware; um serviço autocontido que expõe os dados seria elemento de software.

Provável na prova

- "Defina arquitetura de software" → cite a tripla {Elementos, Organização, Decisões} e pelo menos uma segunda definição.
- "Diferencie padrão, estilo de arquitetura e framework" → use a tabela acima.
- "Quais os 4 elementos essenciais de um padrão de projeto?" → nome, problema, solução, consequências.
- "Framework horizontal × vertical, dê um exemplo de cada."

Módulo 2 · Estilos arquiteturais

Por que estudar

Os dois exercícios da aula 1 são exatamente "dado o cenário, qual estilo e por quê?". É o formato mais natural de questão para este conteúdo, e a justificativa vale tanto quanto o nome do estilo.

A lista de exemplos de estilos

Cliente-servidor · Em camadas · Baseado em componentes · Baseado em regras · Centrado em dados · Orientado a eventos · Orientado a serviços · Objetos distribuídos. Os slides detalham quatro famílias:

Família	Ideia central	Componentes	Quando usar (pista)
Centrada em dados · Repositório	O repositório de dados está no centro ; outros componentes leem ou modificam (atualizam, acrescentam, eliminam) seus dados.	Repositório (armazena/centraliza) + componentes clientes que operam independentes entre si.	Várias ferramentas compartilham os mesmos dados.
Centrada em dados · Quadro negro (Blackboard)	O estado do blackboard dispara a escolha do componente a executar.	Blackboard + base de conhecimento (subsistemas independentes, cada um resolve um aspecto e contribui incrementalmente) + componente de controle (monitora mudanças e decide as ações).	Problema sem algoritmo determinístico, resolvido por especialistas cooperando.
Fluxo de dados · Duto e filtro (pipe and filter)	Filtros conectados por dutos que transmitem dados de um componente para o seguinte.	Cada filtro opera independente , espera entrada em determinada forma e produz saída na forma especificada; desconhece o funcionamento interno dos vizinhos . Linha única = pipeline ou "sequencial por lotes".	"Uma série de transformações aplicadas a uma entrada".
Em camadas	Camadas em diferentes níveis de abstração , cada uma com operações progressivamente mais próximas das instruções de máquina.	Externa: interface do usuário. Intermediárias: serviços utilitários, comunicação, funções de aplicação. Interna: interface com o sistema operacional. Nº de camadas depende das restrições do sistema.	Separar apresentação, negócio e infraestrutura. Anotação sua: "muito comum".
Chamadas e retornos	Estrutura de programa relativamente fácil de modificar e aumentar .	Principal/subprograma : hierarquia de controle, um "principal" invoca componentes que invocam outros. Cliente-servidor : chamadas a procedimentos remotos, invocação a objetos remotos, acesso a serviços distribuídos.	Decomposição funcional; sistemas distribuídos.

Cliente-servidor em detalhe

- É um **modelo para sistemas distribuídos**: representa como dados e processos estão distribuídos entre os componentes.
- **Pode ser implementada em um computador único** (pegadinha clássica).
- Distribui **serviços em servidores específicos**; clientes chamam os serviços; a **rede** permite a comunicação.
- Geralmente integrada por um **middleware**: chamada a procedimentos remotos, invocação a objetos remotos ou acesso a serviços distribuídos.

Exemplo resolvido: exercício 1 da aula 1

"De que forma as aplicações se comunicam em uma arquitetura baseada em repositório de dados? Forneça um exemplo."

Resposta-modelo: de forma **indireta, por meio do repositório**. Os componentes (clientes) operam independentes uns dos outros e não se chamam diretamente: cada um lê ou modifica (atualiza, acrescenta, elimina) os dados centralizados, e é pelo dado gravado que o trabalho de um chega ao outro. Exemplo: um ambiente de desenvolvimento em que editor, compilador, gerador de código e analisador compartilham o mesmo repositório de projeto **EXEMPLO DO SOMMERVILLE**; ou vários sistemas de uma empresa (vendas, estoque, relatórios) sobre o mesmo SGBD.

Exemplo resolvido: exercício 2 da aula 1

Arquivo em linguagem natural → extração e transformação para um padrão estruturado, em módulos especialistas, "a saída de um módulo alimenta o seguinte".

Resposta-modelo: duto e filtro (pipe and filter), na variação **pipeline / sequencial por lotes**, pois o fluxo ocorre em uma única linha de transformações. Justificativa com os termos do slide: (1) o padrão aplica-se "quando se pretende uma série de transformações a uma entrada, realizadas por componentes específicos"; (2) cada módulo especialista é um **filtro**, que espera a entrada em determinada forma e produz a saída na forma especificada; (3) os filtros são independentes e desconhecem o funcionamento interno dos vizinhos, o que permite trocar ou inserir um módulo sem afetar os demais.

Perguntas que o leitor faria

Repositório × blackboard: qual a diferença em uma frase?

No repositório os clientes decidem quando agir e o repositório é passivo. No blackboard há um **componente de controle** que observa o estado dos dados e **escolhe qual especialista executa**; a solução é construída incrementalmente.

Duto e filtro × camadas: os dois não são "uma coisa depois da outra"?

No duto e filtro o **dado flui em um sentido** e é transformado a cada etapa. Em camadas o que existe é **nível de abstração**: a camada de cima pede serviço à de baixo e recebe resposta (vai e volta).

Se o exercício 2 dissesse que os módulos decidem dinamicamente quem atua conforme o estado dos dados?

Aí a resposta mudaria para blackboard. A pista para pipeline é "a saída de um alimenta o seguinte", ordem fixa.

Em que estilo se encaixa o trabalho da disciplina (Vue + Spring Boot + MySQL)?

INFERÊNCIA Cliente-servidor (frontend chama backend pela rede), orientado a serviços (REST) e em camadas no backend (controller → negócio → persistência). Estilos se combinam; o slide diz que o nº de camadas depende "do estilo e/ou combinação de padrões adotados".

O crud em Swing com MySQL é cliente-servidor?

Sim: a aplicação é cliente do servidor MySQL, e o middleware é o driver JDBC. E roda tudo em **localhost**, confirmando que cliente-servidor pode estar em um computador único.

Provável na prova

- Cenário → estilo, com justificativa (os dois exercícios da aula 1, provavelmente reescritos).
- "Quais os componentes do estilo blackboard?" → blackboard, base de conhecimento, controle.
- "Cite as três formas de middleware em cliente-servidor."
- Verdadeiro ou falso: "cliente-servidor exige máquinas diferentes" → falso.

Módulo 3 · GoF: visão geral e padrões de criação

Por que estudar

Sua anotação da aula 2 diz em vermelho: "cai na prova". E o Singleton é o único padrão que você implementou do zero em exercício (DBConnection), então é o candidato natural a "escreva o código".

Design patterns e o GoF

Design patterns: conjunto de regras ou técnicas para projeto e desenvolvimento OO; "melhores práticas" que dão agilidade na solução de problemas recorrentes. Padronizam a representação de componentes, tornam componentes reutilizáveis e **auxiliam a comunicação interna da equipe**.

GoF = "Gang of Four": Erich **Gamma**, Richard **Helm**, Ralph **Johnson**, John **Vlissides**. Livro de **1995**: *Design Patterns: Elements of Reusable Object-Oriented Software*. São **23 padrões em 3 grupos** (sua anotação: "é bom saber"):

Grupo	Qtde	Trata de...	Padrões
Criação	5	como objetos são instanciados	Singleton, Factory Method, Abstract Factory, Builder, Prototype
Estruturais	7	como classes e objetos são <i>compostos</i>	Adapter, Bridge, Composite, Decorator, Façade, Flyweight, Proxy
Comportamentais	11	como objetos <i>interagem</i> e dividem responsabilidade	Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method, Visitor

Os 5 padrões de criação

Padrão	Intenção (slide)	Exemplo da aula	Pista no enunciado
Singleton	Situações onde deve-se ter apenas uma instância de uma classe, acessada por diversos outros objetos; oferece ponto único de acesso .	Window managers, filas de impressão, sistemas de arquivos, conexão com banco . Código: <code>MessageSingleton</code> , <code>DBConnection</code> .	"uma única instância", "acesso global"
Factory Method	Estende funcionalidade pela construção de subclasses sem afetar os clientes. Cliente chama o método de fábrica; subclasses decidem qual classe instanciar , geralmente por um parâmetro recebido.	<code>FactoryPessoa.getPessoa(nome, sexo)</code> devolve <code>Homem</code> ou <code>Mulher</code> (subclasses de <code>Pessoa</code>).	"decidir em runtime qual subclasse criar"
Abstract Factory	Permite criar diferentes ConcreteFactories; cria famílias de objetos relacionados por uma única interface. Objetivos: isolar a criação de objetos de seu uso .	<code>DAOAbstractFactory</code> → <code>JDBCA0Factory</code> / <code>JPADA0Factory</code> (aula 6).	"famílias", "trocar toda a implementação de uma vez"
Builder	Construção de objetos complexos com abordagem passo a passo .	<code>MealDirector</code> + <code>ItalianMealBuilder</code> / <code>JapaneseMealBuilder</code> → <code>Meal</code> .	"montagem em etapas", "mesmo processo, produtos diferentes"
Prototype	Fornecer clones de objetos protótipos que já têm estado inicial predefinido; o cliente customiza a cópia. Esconde como os objetos são criados/copiados.	<code>Employees</code> implements <code>Cloneable</code> , com <code>clone()</code> copiando a lista.	"copiar um objeto caro de criar"

Singleton por dentro

O slide diz que ele permite: (1) garantir que apenas uma instância é criada; (2) fornecer um único ponto de acesso; (3) permitir que haja **múltiplas instâncias no futuro sem afetar os clientes**, e nesse caso ele "se transforma numa Factory". Os três ingredientes que não podem faltar no código:

Como fica no código (slide da aula 2)

```
public class ClassicSingleton {
    private static ClassicSingleton instance = null; // 1) guarda a única instância, na classe
    private ClassicSingleton() { } // 2) construtor privado: ninguém de fora dá new
    public static ClassicSingleton getInstance() { // 3) ponto único de acesso, estático
        if (instance == null) { // cria só na primeira chamada (lazy)
            instance = new ClassicSingleton();
        }
        return instance;
    }
}
```

Exemplo resolvido: o que imprime o MessageSingletonTest?

`getInstance("First").getMessage()` e depois `getInstance("Second").getMessage()`.

Saída: "First" e "First". Na segunda chamada `instance` já não é `null`, então o construtor não roda de novo e o parâmetro "Second" é ignorado. É a prova de que existe uma única instância.

Builder: os 4 componentes

Componente	Papel (slide)	No exemplo Meal
Builder	Interface ou classe abstrata para a criação das partes (passos) de um produto.	<code>MealBuilder</code> : <code>buildDrink()</code> , <code>buildMainCourse()</code> , <code>buildSide()</code> , <code>getMeal()</code>
Concrete Builder	Constrói e monta as partes do produto, implementando a builder.	<code>ItalianMealBuilder</code> (red wine, pizza, bread), <code>JapaneseMealBuilder</code> (sake, chicken teriyaki, miso soup)
Director	Contém um builder e controla o algoritmo que gera o produto final; recebe o Concrete Builder por parâmetro.	<code>MealDirector.constructMeal()</code> chama os 3 passos em ordem
Product	O objeto complexo que está sendo construído.	<code>Meal</code>

Exemplo resolvido: Prototype

`emps` carrega [Pankaj, Raj, David, Lisa]. `empsNew = emps.clone()` e adiciona "John"; `empsNew1 = emps.clone()` e remove "Pankaj".

Saída: `emps` = [Pankaj, Raj, David, Lisa] (intacto) · `empsNew` = [Pankaj, Raj, David, Lisa, John] · `empsNew1` = [Raj, David, Lisa]. O original não muda porque o `clone()` cria uma **nova lista** e copia os itens; se devolvesse a mesma referência, os três seriam alterados juntos.

Perguntas que o leitor faria

Factory Method × Abstract Factory?

Factory Method é **um método** que decide qual subclasse de *um* produto criar (Pessoa → Homem ou Mulher). Abstract Factory é **uma fábrica de fábricas**: uma interface com métodos de criação, e cada fábrica concreta produz uma *família* coerente (tudo JDBC ou tudo JPA). No crud, `getFactory(tipo)` escolhe a fábrica; `createProdutoDAO()` é o método de fábrica dentro

dela.

Builder x Factory?

A factory entrega o objeto pronto em uma chamada. O Builder monta em **vários passos** coordenados por um Director, e o mesmo roteiro de passos gera produtos diferentes conforme o Concrete Builder.

Por que o atributo instance é static?

Porque precisa existir **antes de qualquer objeto** e ser compartilhado por todos: pertence à classe. Pelo mesmo motivo `getInstance()` é estático: sem instância, não há como chamar método de instância.

O Singleton do slide é seguro com threads?

FORA DO SLIDE Não: duas threads podem passar pelo `if (instance == null)` ao mesmo tempo e criar duas instâncias. A correção usual é `synchronized` no `getInstance()`. O professor não cobrou isso, mas é um bom comentário de "consequência".

O exercício pediu `finalize()` como "destrutor". Isso é confiável?

FORA DO SLIDE Na prova, responda como na aula: reimplementa-se `finalize()` de `Object` para fechar `Statement` e `Connection`. Na prática o Java não garante quando (ou se) o `finalize()` roda, e ele está obsoleto nas versões recentes.

Provável na prova

- "Escreva um Singleton em Java" ou "complete o código" (faltando `private`, `static` ou o `if`).
- "Quantos padrões GoF existem e como são classificados? Cite dois de cada grupo."
- Múltipla escolha estilo concurso: "qual alternativa só tem padrões de criação?"
- "Quais os componentes do Builder?" → Builder, Concrete Builder, Director, Product.

Módulo 4 · GoF: padrões estruturais

Por que estudar

São os 7 padrões que mais se confundem entre si, porque quase todos "embrulham" um objeto em outro. A prova cobra a *intenção*: o que muda é o **motivo** do embrulho.

Padrão	Intenção (slide)	Exemplo da aula	Pista no enunciado
Adapter	Realiza adaptações em objetos e métodos para integração entre eles. Útil para integrar com código de terceiros ; um objeto se adapta à interface de outro, de pacote não relacionado.	<code>SortListAdapter implements Sorter</code> : cliente usa <code>int[]</code> , mas o <code>NumberSorter</code> de terceiros só aceita <code>List<Integer></code> . O adapter converte ida e volta.	"interfaces incompatíveis", "código de terceiros", "legado"
Bridge	Desacopla uma abstração de sua implementação ; esconde detalhes de implementação; permite extensibilidade. Feito por composição : o cliente mantém instância de um Implementor (abstrato/interface) e a bridge direciona as requisições ao <code>ConcretImplementor</code> .	<code>Shape</code> (abstração: <code>Triangle</code> , <code>Pentagon</code>) contém um <code>Color</code> (implementor: <code>RedColor</code> , <code>GreenColor</code>).	"duas hierarquias que variam independentemente"
Composite	Organiza objetos em estrutura de árvore para representar hierarquias/taxonomias. Uniformiza o tratamento de objetos individuais e composições. Usado quando há definição recursiva .	<code>Drawing implements Shape</code> e contém <code>List<Shape></code> ; <code>draw()</code> repassa a todos os filhos.	"árvore", "parte-todo", "composto por outros"
Decorator	Em tempo de execução , modifica e incrementa a funcionalidade de um objeto , sem afetar outras instâncias da mesma classe. Por herança ou composição.	<code>new SportsCar(new LuxuryCar(new BasicCar()))</code>	"adicionar responsabilidades dinamicamente", "sem criar subclasse para cada combinação"
Façade	Simplifica o acesso a APIs complexas : interface simplificada e de alto nível para um subsistema; integra o acesso em um ponto único .	<code>TravelFacade.getFlightsAndHotels(from, to)</code> esconde <code>FlightBooker</code> e <code>HotelBooker</code> .	"simplificar", "subsistema complexo", "um ponto de entrada"
Flyweight	Cria grande número de objetos de maneira eficiente; evita custo e desperdício de memória ao criar o mesmo objeto muitas vezes; objetos em memória são reutilizados.	<code>ShapeFactory.getShape(tipo)</code> guarda as formas já criadas e devolve a mesma instância.	"milhares de objetos parecidos", "economia de memória", "cache"
Proxy	Fornecer um substituto ou intermediário de acesso a um objeto; controla o acesso . Geralmente implementa a mesma interface , intercepta a chamada, checa permissões e delega ao real.	<code>CommandExecutorProxy</code> : só admin pode executar comandos que começam com <code>rm</code> .	"controle de acesso", "permissão", "intermediário transparente"

Os pares que confundem

Par	Como separar
-----	--------------

Adapter × Façade	Adapter converte uma interface em outra que o cliente espera (o problema é incompatibilidade). Façade cria uma interface nova e mais simples sobre várias classes (o problema é complexidade).
Proxy × Decorator	Ambos implementam a mesma interface do objeto embrulhado. Proxy controla/restringe o acesso; Decorator acrescenta comportamento. No Proxy da aula o objeto real é criado <i>dentro</i> do proxy; no Decorator ele vem <i>de fora</i> , pelo construtor, e pode ser empilhado.
Bridge × Adapter	Bridge é planejada antes , para que abstração e implementação evoluam separadas. Adapter é remendo depois , para fazer duas coisas prontas conversarem.
Composite × Decorator	Composite agrega vários filhos (árvore). Decorator embrulha um objeto (corrente).
Flyweight × Singleton	Singleton: uma instância da classe. Flyweight: poucas instâncias compartilhadas , uma por estado intrínseco (um Line, um Oval...), servidas por uma fábrica com cache.

Exemplo resolvido: saída do Decorator

```
Car c = new SportsCar(new LuxuryCar(new BasicCar())); c.assemble();
```

Cada decorator chama `super.assemble()` (que delega ao carro embrulhado) e **depois** imprime o seu texto. A chamada desce até o `BasicCar` e volta: **"Basic Car. Adding features of Luxury Car. Adding features of Sports Car."** A ordem de impressão é de dentro para fora.

Exemplo resolvido: saída do Proxy

```
new CommandExecutorProxy("Pankaj", "wrong_pwd") → senha errada, isAdmin = false . runCommand("ls -ltr")
```

executa normalmente. `runCommand("rm -rf abc.pdf")`: o proxy faz `trim()`, vê que começa com "rm" e lança a exceção **"rm command is not allowed for non-admin users."** O objeto real nunca é chamado.

Como fica no código: Bridge e Composite lado a lado

```
// BRIDGE: a abstração (Shape) TEM UM implementor (Color), recebido por composição
public abstract class Shape {
    protected Color color; // a "ponte" para a outra hierarquia
    public Shape(Color c) { this.color = c; }
    abstract public void applyColor();
}
Shape tri = new Triangle(new RedColor()); // 2 formas x 2 cores sem criar 4 subclasses

// COMPOSITE: o composto É UM Shape e TEM VÁRIOS Shape
public class Drawing implements Shape {
    private List<Shape> shapes = new ArrayList<>();
    public void draw(String fillColor) {
        for (Shape sh : shapes) sh.draw(fillColor); // trata folha e composto do mesmo jeito
    }
    public void add(Shape s) { shapes.add(s); } // um Drawing pode conter outro Drawing
}
```

Perguntas que o leitor faria

Por que a Bridge evita "explosão de subclasses"?

Sem ela, 2 formas × 2 cores = 4 classes (TriânguloVermelho, TriânguloVerde...); com 5 formas e 6 cores seriam 30. Com Bridge são 5 + 6 = 11, e adicionar uma cor não toca nas formas.

O DAO da aula 5 é uma Façade?

INFERÊNCIA Tem cara de Façade (esconde JDBC, SQL e ResultSet atrás de `create`, `retrieveAll` ...), mas os slides o tratam como padrão J2EE próprio. Em GRASP, ele é *Pure fabrication + Indirection*.

"Estado intrínseco" no Flyweight é o quê?

É a parte do objeto que pode ser **compartilhada** (ex.: o tipo da forma). O que varia a cada uso (posição, cor naquele desenho) é passado por parâmetro na hora de usar.

Swing usa algum desses?

INFERÊNCIA Sim: `JPanel` contendo componentes que podem ser outros `JPanel` é Composite; `JScrollPane` embrulhando um componente é Decorator.

Provável na prova

- Exercício 1 da lista 2 da aula 3: "taxonomia/árvore, artefato composto por outros" → **Composite**; "integração com códigos de terceiros" → **Adapter**.
- "Diferencie Proxy de Decorator" ou "Adapter de Façade".
- Dado um trecho como `new A(new B(new C()))`, identificar o padrão (Decorator) e a saída.

Módulo 5 · GoF: padrões comportamentais

Por que estudar

É o maior grupo (11) e o que gerou mais exercício: a lista 1 da aula 3 inteira é Observer com eventos em Swing, e a lista 2 é "requisito → padrão" mais uma questão de concurso sobre este grupo.

Padrão	Intenção (slide)	Exemplo da aula	Pista no enunciado
Chain of Responsibility	Desacopla o remetente do recipiente de uma requisição; interpõe uma cadeia de objetos entre eles.	Caixa eletrônico: <code>Dollar50Dispenser</code> → <code>Dollar20</code> → <code>Dollar10</code> , cada um trata o que pode e passa o resto.	"vários tratadores em sequência", "passa adiante"
Command	Faz requisições a objetos sem saber o destinatário nem os detalhes da operação; desacopla quem pede de quem realiza. Similar a callbacks em linguagens procedurais.	<code>RemoteControl.setCommand(new LightOnCommand(light));</code> <code>pressButton()</code>	"encapsular uma ação como objeto", "botões", "desfazer/fila de ações"
Interpreter	Define representações gramaticais para linguagens e provê interpretadores para cada gramática.	Slide: JVM (interpreta bytecode), Google Translator . Código: <code>IntToBinaryExpression</code> , <code>IntToHexExpression</code> .	"gramática", "linguagem", "expressões"
Iterator	Forma padrão de percorrer grupos de objetos em uma agregação. Muito usado no Java Collection Framework .	<code>ChannelIterator</code> : <code>hasNext()</code> , <code>next()</code> , filtrando por tipo de canal.	"percorrer sem expor a estrutura interna"
Mediator	Usa um objeto para coordenar mudanças de estado entre outros. Funciona como hub de comunicação , "many-to-many"; diminui acoplamento; lógica mais coesa.	Chat: <code>UserImpl.send()</code> chama <code>mediator.sendMessage()</code> , que entrega a todos os outros usuários.	"muitos objetos conversando entre si", "centralizar a comunicação"
Memento	Armazena e recupera o estado anterior de um objeto: desfazer operação, recuperar de estado inválido. O estado fica protegido do acesso externo em um objeto memento.	<code>FileWriterUtil.save()</code> devolve um <code>Memento</code> (classe interna privada); <code>FileWriterCaretaker</code> guarda e faz <code>undo</code> .	"undo", "snapshot", "restaurar"
Observer	Estabelece relações causa-efeito entre objetos sem ligação direta. Cada Observer é notificado sobre alterações de estado do objeto observado (Sujeito).	Slide: AWT, JavaBeans, <code>javax.naming</code> , I/O handlers. Java adere com o modelo de eventos (Listeners). Código: <code>Origem</code> / <code>Ouvinte</code> .	"notificar interessados que se cadastram", "eventos"
State	Altera o comportamento de um objeto com base em seu estado atual . Um Context mantém referência a uma implementação de State.	<code>TVContext.setState(new TVStartState());</code> <code>doAction()</code> → "TV is turned ON".	"comportamento depende do estado", "máquina de estados sem if/else"
Strategy	Define um conjunto de algoritmos alternáveis , encapsulados em classes "irmãs" (mesma interface). Seleciona o algoritmo pelo objeto passado; pode variar com o tempo.	<code>cart.pay(new PaypalStrategy(...))</code> ou <code>cart.pay(new CreditCardStrategy(...))</code>	"família de algoritmos intercambiáveis", "escolher em runtime"

Template Method	Define os passos de um algoritmo ; implementa a parte genérica e deixa a específica para as subclasses . O método template geralmente é final , para que subclasses não mudem o algoritmo base.	<code>HouseTemplate.buildHouse()</code> : fundação → pilares → paredes → janelas. <code>WoodenHouse</code> e <code>GlassHouse</code> só implementam pilares e paredes.	"sequência obrigatória de passos", "esqueleto fixo, passos customizáveis"
Visitor	Separa a estrutura de uma coleção de objetos das operações realizadas nela. Cria nova funcionalidade sem alterar as classes . Limitação : adicionar novas classes à coleção obriga a modificar os Visitors.	<code>car.accept(new CarElementPrintVisitor())</code> ; cada elemento faz <code>visitor.visit(this)</code> .	"nova operação sobre uma estrutura estável"

Observer: o padrão que virou exercício

A lista 1 da aula 3 pede quatro peças. Decore o papel de cada uma, porque é o mesmo desenho de todo Listener do Java:

Peça	Papel no padrão	Detalhe
<code>MessageEvent</code> extends <code>EventObject</code>	O evento : carrega a informação da mudança.	Construtor recebe a <code>source</code> ; tem <code>get/setMessage</code> .
<code>MessageEventListener</code> (interface)	O contrato do Observer .	Um método: <code>messageArrived(MessageEvent e)</code> .
<code>Origem</code> (JFrame)	O Sujeito : mantém a lista de ouvintes, cadastra e notifica.	<code>addMessageListener(mel)</code> ; ao clicar em enviar: cria o evento, itera a lista e chama <code>messageArrived</code> em cada um.
<code>Ouvinte</code> (JFrame)	Observer concreto .	<code>implements MessageEventListener</code> ; exibe <code>e.getMessage()</code> na caixa de texto. Várias instâncias.

Como fica no código (núcleo do exercício 1 da aula 3)

```
// SUJEITO
private List<MessageEventListener> lista = new ArrayList<>();
public void addMessageListener(MessageEventListener mel) { lista.add(mel); } // cadastro

private void enviar() {
    MessageEvent me = new MessageEvent(this); // this = a origem do evento
    me.setMessage(jTextField1.getText());
    for (MessageEventListener mel : lista) { // notifica TODOS os cadastrados
        mel.messageArrived(me); // a Origem não sabe que são janelas
    }
}

// OBSERVER
public class Ouvinte extends JFrame implements MessageEventListener {
    public void messageArrived(MessageEvent e) { jTextField1.setText(e.getMessage()); }
}
```

Os exercícios 2 e 3 da mesma lista (bolinha movida pelas setas; pintar pixels com click e drag) são o mesmo padrão com Listeners prontos do Swing: `KeyListener` e `MouseListener` / `MouseMotionListener`. O painel é o Sujeito; sua classe é o Observer.

Os pares que confundem

Par	Como separar
-----	--------------

Strategy × State	Mesmo desenho (contexto + interface + implementações). Em Strategy o cliente escolhe o algoritmo (<code>cart.pay(estrategia)</code>). Em State o comportamento muda porque o estado do objeto mudou (<code>context.setState</code>), representando uma situação do objeto, não uma opção de algoritmo.
Strategy × Template Method	Strategy troca o algoritmo inteiro , por composição . Template Method fixa o esqueleto e troca alguns passos , por herança .
Observer × Mediator	Observer: um sujeito → muitos observadores , comunicação em um sentido. Mediator: muitos ↔ muitos passando por um hub que conhece todos.
Command × Chain of Responsibility	Command empacota a ação em um objeto com <code>execute()</code> . Chain passa a requisição por vários candidatos até alguém tratar. Ambos desacoplam remetente e receptor.
Template Method × Builder	Ambos têm "passos". Template Method é sobre comportamento (algoritmo com passos abstratos, herança). Builder é sobre criação (montar um produto, com Director + composição).

Exemplo resolvido: Chain of Responsibility com saque de 130

Cadeia: 50 → 20 → 10. **Dollar50**: $130 \geq 50 \rightarrow 130 / 50 = 2 \text{ notas}$, resto 30 → passa 30 adiante. **Dollar20**: $30 \geq 20 \rightarrow 1 \text{ nota}$, resto 10 → passa 10. **Dollar10**: $10 \geq 10 \rightarrow 1 \text{ nota}$, resto 0 → fim.

Saída: "Dispensing 2 50\$ note", "Dispensing 1 20\$ note", "Dispensing 1 10\$ note". Com 125 o programa nem entra na cadeia: `amount % 10 != 0` encerra com "Amount should be in multiple of 10s."

Exemplo resolvido: lista 2 da aula 3

1a) "sequência obrigatória de passos, generalista, passos implementados para cada cliente" → **Template Method**. **1b)** "taxonomia em árvore, artefato composto por outros" → **Composite**. **1c)** "integração com códigos de terceiros" → **Adapter**. **1d)** "comunicar alterações de estado a quaisquer interessados que se cadastrem" → **Observer**.

2) Comportamentais: alternativa **a) Command, Observer e Iterator**. As demais misturam grupos: b) Singleton é de criação; c) Prototype é de criação, Adapter e Bridge são estruturais; d) Builder é de criação; e) Proxy e Decorator são estruturais, Singleton é de criação.

Perguntas que o leitor faria

Por que o `buildHouse()` é final?

Para impedir que uma subclasse sobrescreva o método template e mude a ordem (ex.: janelas antes da fundação). A subclasse só pode preencher os passos abstratos.

No exemplo Template Method, quais passos são fixos e quais variam?

`buildFoundation()` e `buildWindows()` são `private` na classe base (fixos). `buildPillars()` e `buildWalls()` são `abstract` (variavam: madeira ou vidro).

Qual a limitação do Visitor, em termos práticos?

Se surgir uma nova classe de elemento (ex.: `Door`), é preciso adicionar `visit(Door)` na interface `CarElementVisitor` e em **todos** os visitors existentes. Por isso ele só vale para estruturas estáveis.

Memento não quebra o encapsulamento ao guardar o estado fora do objeto?

Não: no exemplo, `Memento` é classe **interna e privada** de `FileWriterUtil`. O Caretaker guarda o objeto como `Object`, sem conseguir ler seu conteúdo.

Onde o Observer reaparece na matéria?

Em GRASP (é citado como especialização de Indirection) e em MVC (o Model notifica a View por Observer). É o padrão mais "reutilizado" pelo professor.

Provável na prova

- "Requisito → padrão" (como o exercício 1) com outros padrões: undo → Memento; algoritmos intercambiáveis → Strategy; hub de comunicação → Mediator; controle de acesso → Proxy.
- Múltipla escolha de classificação (como o exercício 2).
- "Explique o Observer e dê um exemplo em Java" → Sujeito, Observers, notificação; Listeners do AWT/Swing.
- "Qual a limitação do Visitor?" / "Por que o método template é final?"

Módulo 6 · GRASP: a quem dar cada responsabilidade

Por que estudar

Sua anotação da aula 4: "aula importante pra prova, conteúdo pesado". GRASP é a lista mais longa (9 nomes) e o exercício 1 da aula já mostra o formato: um cenário de código ruim e a pergunta "qual princípio foi ferido? Por quê?".

GRASP = General Responsibility Assignment Software Patterns. Craig Larman, 1997. Conjunto de **9 princípios fundamentais**, aplicados ao projeto OO e à **atribuição de responsabilidades**. Cada um responde a uma pergunta de projeto:

Princípio	Pergunta que responde	O que diz (slide)	Exemplo do slide
Information expert	Quem deve fazer isso?	Atribuir a responsabilidade levando em conta o tipo de informação que a classe contém .	Simulação a 600 fps e tela que não suporta: <code>SpeedControl</code> tem as informações de quadros exibidos/faltantes, então é ela que decide quando exibir o quadro.
Creator	Quem deve criar A?	B cria A se: B contém A · B agrega A · B tem os dados de inicialização de A · B registra A · B faz uso intenso/coeso de A.	—
Controller	Quem recebe a ação vinda da UI?	Provê o encaminhamento de uma ação ; encapsula uma operação do sistema; cria uma camada entre UI e classes de domínio , o que permite trocar o tipo de UI (linha de comando, Web, GUI).	Classe <code>Map</code> : o usuário move o cursor na UI, a ação é delegada ao <code>Map</code> , que assume o controle e chama os objetos de domínio.
Indirection	Como evitar que dois objetos se conheçam?	Intermedia a interação entre dois objetos, mantendo o acoplamento baixo; papéis ficam isolados. Adapter, Façade e Observer são especializações deste princípio.	—
Low coupling	Como reduzir o impacto de mudanças?	Acoplamento mede quanto uma classe está conectada a, ou depende de, outras . O princípio procura reduzir a dependência para diminuir o impacto de mudanças e aumentar a reutilização .	Formas de acoplamento na tabela abaixo.
High cohesion	Como manter classes focadas e compreensíveis?	Coesão mede quão relacionadas ou focadas estão as responsabilidades de um elemento. Alta coesão: poucas tarefas, nº relativamente pequeno de métodos, funcionalidade altamente relacionada; mais estável, fácil de entender, manter e reusar; permite projeto modular .	—
Polymorphism	Como tratar variações por tipo?	Característica fundamental da POO; o princípio recomenda seu uso no lugar de testar o tipo do objeto (que tem alto custo em runtime). Chama-se o método pelo supertipo e a implementação especializada é acionada.	Dois dispositivos, <code>USB</code> e <code>Network</code> , com leitura chamada de forma geral.
Protected variants	Como blindar o sistema contra o que vai variar?	Estabilizar o sistema perante variações: identificar pontos de variação e criar uma interface estável em volta deles (encapsulamento, interfaces, polimorfismo, indireção, padrões). Evitar enviar mensagens a objetos muito distantes.	Máquinas virtuais e brokers são motivados por este princípio.

Pure fabrication	Onde pôr o que não é do domínio?	Atribuir responsabilidades " acessórias " (não do domínio) a classes artificiais criadas para isso, mantendo alta coesão, baixo acoplamento e reuso.	Um conjunto de classes que realizam persistência em um BD .
-------------------------	----------------------------------	---	--

As quatro formas de acoplamento (Low coupling)

Forma	Em Java
Uma classe contém outra, como atributo	<code>private Log log;</code>
Uma classe obtem a instância de outra (por construção ou retorno)	<code>new Log("Arquivo.txt") · DBConnection.getInstance()</code>
Uma classe estende outra	<code>class Homem extends Pessoa</code>
Uma classe implementa uma interface	<code>class JDBCProdutoDAO implements ProdutoDAO</code>

Frase do slide que relaciona os dois: "**quanto menor acoplamento, maior é a coesão**". Objetivo de projeto: **baixo acoplamento + alta coesão**.

Exemplo resolvido: exercício 1 da aula 4

"Classe com muitas funcionalidades, código enorme; várias nem deveriam estar relacionadas a ela, o que a tornou complicada de entender e usar. Qual princípio GRASP ele feriu? Por quê?"

Resposta-modelo: feriu o princípio da **alta coesão (High cohesion)**. Coesão mede quão relacionadas e focadas estão as responsabilidades de um elemento; para ser alta, a classe deve ter poucas tarefas, um número relativamente pequeno de métodos e funcionalidades altamente relacionadas. A classe descrita acumula responsabilidades não relacionadas, por isso ficou difícil de entender, manter e reusar, que são justamente os benefícios que a alta coesão traria. Correção: distribuir as funcionalidades para as classes que detêm a informação (*Information expert*) e mover o que não é do domínio para classes artificiais (*Pure fabrication*).

Se a pergunta fosse sobre SOLID, a resposta equivalente seria *Single-responsibility*. Leia qual família o enunciado pede.

Exemplo resolvido: GRASP dentro do crud

INFERÊNCIA

Pure fabrication: `DBConnection`, `JDBCProdutoDAO` e as fábricas não existem no domínio "produto"; foram fabricadas para tirar a persistência da janela. **Indirection:** o DAO fica entre a janela e o banco. **Protected variants:** o ponto de variação é "como acessar o BD" e a interface estável é `ProdutoDAO`. **Polymorphism:** a janela chama `create()` pelo supertipo `ProdutoDAO`, sem `if (tipo == JDBC)`. **Creator:** `JDBCDAOFactory` cria `JDBCProdutoDAO` porque tem os dados para decidir e inicializar. **Low coupling:** a janela depende só de `ProdutoDAO`, `Produto` e da fábrica abstrata.

Perguntas que o leitor faria

Controller do GRASP é o Controller do MVC?

É a mesma ideia: um objeto que não é UI recebe a ação do usuário e a encaminha ao domínio. O MVC é uma arquitetura que aplica esse princípio de forma sistemática. O benefício declarado nos dois casos é poder trocar a UI.

Indirection × Pure fabrication?

Indirection diz "**ponha um intermediário**" entre dois objetos. Pure fabrication diz "**invente uma classe que não é do domínio**" para uma responsabilidade acessória. Muitas vezes o intermediário é uma classe fabricada (o DAO é os dois).

Protected variants × Open-closed (SOLID)?

São a mesma preocupação vista de dois lados: proteger o sistema do que varia, por meio de uma interface estável, é o que permite estender sem modificar.

Se implementar interface também é acoplamento, por que usamos tanto?

Porque é o acoplamento **mais fraco**: depende-se de um contrato estável, não de uma implementação. O objetivo é baixo acoplamento, não zero.

Como decorar os 9?

Três grupos: **quem faz** (Information expert, Creator, Controller) · **qualidade** (Low coupling, High cohesion) · **técnicas para chegar lá** (Indirection, Polymorphism, Protected variants, Pure fabrication).

Provável na prova

- Cenário → princípio ferido, com justificativa (exercício 1).
- "O que significa GRASP? Cite e explique 3 princípios."
- "Quais padrões GoF são especializações de Indirection?" → Adapter, Façade e Observer.
- "Em que condições a classe B deve criar a classe A?" → as 5 condições do Creator.
- "Defina acoplamento e coesão e a relação entre eles."

Módulo 7 · SOLID

Por que estudar

O exercício 4 da aula 4 é literalmente "Explique cada um dos 5 princípios SOLID". É uma questão dissertativa pronta, com resposta fechada. Não pode perder ponto aqui.

SOLID: princípios para programação orientada a objetos. **Robert C. Martin, 2000**. Acrônimo dos 5 princípios. Objetivo: tornar o projeto de software mais **compreensível, flexível e fácil de manter**.

	Princípio	Enunciado (slide)	Detalhes que valem ponto
S	Single-responsibility	Uma classe deve ter apenas uma responsabilidade : apenas uma razão para mudar .	Vale para módulo, classe ou função; deve encapsular essa parte e estar estreitamente alinhado a ela.
O	Open-closed	Classes (módulos, funções) devem ser abertas para extensão, mas fechadas para modificação .	Estende-se o comportamento sem modificar o código-fonte , por herança e polimorfismo . Aberto = ainda disponível para extensão (adicionar campos, funções). Fechado = disponível para uso por outros módulos, com descrição estável e bem definida (a interface).
L	Liskov substitution	Se S é subtipo de T, objetos do tipo T podem ser substituídos por objetos do tipo S sem alterar as propriedades desejáveis do programa.	Contrato do subtipo comportamental: pré-condições não podem ser reforçadas · pós-condições não podem ser enfraquecidas · invariantes preservados · encapsulamento mantido.
I	Interface segregation	Nenhum cliente deve ser forçado a depender de métodos que não usa .	Dividir interfaces grandes em menores e mais específicas ("interfaces de função"). Mantém o sistema desacoplado, mais fácil de refatorar, alterar e reimplantar. Semelhante à alta coesão (GRASP) .
D	Dependency inversion	Módulos de alto nível não devem depender de módulos de baixo nível ; ambos devem depender de abstrações . Abstrações não dependem de detalhes; detalhes dependem de abstrações .	Objetos de alto e baixo nível dependem da mesma abstração (interfaces). Permite acoplamento fraco (GRASP) .

As pontes SOLID ↔ GRASP que o próprio slide faz

SOLID	GRASP	Onde está dito
Interface segregation	High cohesion	"É semelhante ao princípio de alta coesão (GRASP)"
Dependency inversion	Low coupling	"Permite acoplamento fraco (GRASP)"
Single-responsibility	High cohesion	INFERÊNCIA mesma ideia: poucas responsabilidades, bem relacionadas
Open-closed	Protected variants / Polymorphism	INFERÊNCIA o slide de OCP cita "herança e polimorfismo"

Exemplo resolvido: SOLID lido no crud da aula 6

INFERÊNCIA

S: `Produto` só guarda dados; `JDBCProdutoDAO` só fala SQL; `CrudProduto` só cuida da tela. Antes (POO), a janela fazia as três coisas.

O: para suportar JPA foi criada `JPAProdutoDAO`, sem alterar uma linha de `JDBCProdutoDAO` nem de `ProdutoDAO`: estendeu-se sem modificar.

L: a janela recebe um `ProdutoDAO` e funciona igual com a implementação JDBC ou JPA. Se `JPAProdutoDAO.delete()` exigisse algo que a interface não promete (pré-condição reforçada), a substituição quebraria.

I: `ProdutoDAO` tem só os 5 métodos de CRUD de produto. Uma interface única "`TudoDAO`" com métodos de cliente, fatura e produto forçaria cada implementação a depender do que não usa.

D: a janela (alto nível) e o `JDBCProdutoDAO` (baixo nível) dependem ambos da abstração `ProdutoDAO`. A linha trocada no exercício mostra a inversão: de `new JDBCProdutoDAO()` para `DAOAbstractFactory.getFactory(...).createProdutoDAO()`.

Resposta-modelo compacta para o exercício 4

S, responsabilidade única: cada classe tem uma só responsabilidade e, portanto, um só motivo para mudar. **O**, aberto-fechado: entidades abertas para extensão e fechadas para modificação; novo comportamento entra por herança/polimorfismo, sem mexer no código existente. **L**, substituição de Liskov: um subtipo deve poder substituir seu supertipo sem alterar as propriedades desejáveis do programa (não reforça pré-condições, não enfraquece pós-condições, preserva invariantes). **I**, segregação de interfaces: nenhum cliente deve depender de métodos que não usa; prefira várias interfaces pequenas e específicas. **D**, inversão de dependência: módulos de alto nível não dependem dos de baixo nível, ambos dependem de abstrações; detalhes dependem de abstrações, não o contrário.

Perguntas que o leitor faria

Como uma classe pode ser aberta e fechada ao mesmo tempo?

São dois públicos diferentes. Para quem **usa** a classe ela está fechada: interface estável, não muda. Para quem precisa de **novo comportamento** ela está aberta: cria-se uma subclasse ou nova implementação da interface.

Um exemplo clássico de violação de Liskov?

FORA DO SLIDE `Quadrado extends Retangulo`: quem recebe um `Retangulo` espera que `setLargura(5)` não altere a altura (pós-condição). O quadrado altera, enfraquecendo a pós-condição; o programa que funcionava com T deixa de funcionar com S.

Dependency inversion é o mesmo que injeção de dependência?

Não. DIP é o **princípio** ("dependa de abstrações"). DI é uma **técnica** para entregar a implementação concreta ao objeto sem que ele a crie (módulo 8). DI é uma forma de realizar DIP.

"Inversão" de quê, afinal?

No projeto ingênuo a seta de dependência vai do alto nível para o baixo (janela → JDBC). Ao introduzir a interface, o módulo de baixo nível passa a **apontar para a abstração** (implementa `ProdutoDAO`), e a abstração pertence ao nível alto. A seta do detalhe se inverteu.

Provável na prova

- "Explique cada um dos 5 princípios SOLID" (exercício 4, quase certo em alguma forma).
- "O que significa cada letra?" + autor e ano (Robert C. Martin, 2000).
- "Quais as restrições de contrato para um subtipo comportamental?"
- Trecho de código com uma interface gigante ou uma classe faz-tudo → qual princípio é violado.

Módulo 8 · Princípio de Hollywood, IoC, DI e MVC

Por que estudar

Dois dos quatro exercícios da aula 4 saem daqui ("o que é MVC...", "o que é injeção de dependências..."), ambos pedindo **definição + exemplo + vantagem**. Além disso, IoC/DI é o que o Spring faz, e MVC é a arquitetura do trabalho.

Princípio de Hollywood

"Não ligue para nós, nós ligaremos para você." Similar à inversão de dependências, porém especifica **como** o componente de nível superior e os de nível inferior interagem **sem criar dependências**. Permite dependências em **direção única**: o componente de alto nível chama os de baixo nível de modo que nenhum dependa do outro.

IoC x DI

	Inversão de Controle (IoC)	Injeção de Dependências (DI)
O que é	Princípio de programação que inverte o fluxo de controle tradicional (imperativo) .	Passagem de uma dependência (um objeto usado como serviço) para o objeto cliente que a usaria.
Como	Em vez de programar todo o fluxo da aplicação, gera-se código para tarefas específicas ; a infraestrutura (container) chama essas funções quando oportuno. A infraestrutura provê o laço de eventos e mensagens; a aplicação só trata os eventos.	Em vez de o cliente especificar qual serviço/objeto será usado, a estrutura faz isso para ele .
Ganho	Aumenta a modularidade e a extensibilidade .	Separation of Concerns : o cliente não se preocupa com localização e instância dos serviços; a estrutura sim. Integra dependências em camadas de abstração compartilhadas.
Relação	IoC é o princípio geral ("quem manda no fluxo é o container"). DI é a aplicação dele à criação de dependências ("quem dá <code>new</code> não sou eu").	

Como fica no código (exemplo do slide, antes e depois)

```
// ANTES: a classe cria sua própria dependência
public class VendaDeProduto {
    public void vendeProduto(Produto produto) {
        // todo o código para a venda do produto...
        Log log = new Log("Arquivo.txt");    // problema: não seria responsabilidade dela criar o Log
        log.grava(produto);
    }
}

// DEPOIS: a dependência é INJETADA pelo construtor
public class VendaDeProduto {
    private Log log;
    public VendaDeProduto(Log logVenda) {    // quem instancia VendaDeProduto entrega o Log pronto
        this.log = logVenda;
    }
    public void vendeProduto(Produto produto) {
        // todo o código para a venda do produto...
        log.grava(produto);                  // a classe somente USA a instância injetada
    }
}
```

Atenção a um detalhe do slide: lá o construtor aparece como `public void VendaDeProduto(Log logVenda)`. Com `void`, o Java trata isso como um *método* comum, não como construtor, e o `log` ficaria `null`. O texto do slide diz "injeção de dependência **no construtor**", então na prova escreva sem o `void`, como acima.

Resposta-modelo: exercício 3 da aula 4 (o que é DI, exemplo, vantagem)

O que é: uma dependência é um objeto que pode ser usado como serviço; injeção é a passagem dessa dependência ao objeto cliente que a usaria. Em vez de o cliente especificar e criar o serviço, a estrutura (container/framework ou quem o instancia) faz isso por ele.

Exemplo: `VendaDeProduto` deixava de fazer `new Log("Arquivo.txt")` e passa a receber um `Log` no construtor, apenas usando a instância injetada.

Vantagem: separação de interesses e baixo acoplamento. A classe não precisa saber onde está nem como se cria o serviço; pode-se trocar o `Log` em arquivo por um log em banco, ou por um falso em testes, sem alterar `VendaDeProduto`.

MVC

Padrão **amplamente utilizado**, semelhante a uma ideia antiga do **Smalltalk**. É uma **divisão de responsabilidades**:

Parte	Representa	Palavra-chave
Model	O estado da aplicação	dados
View	A camada de apresentação	exibição
Controller	O fluxo de eventos	ações

Fluxo do diagrama do slide: as **ações do usuário** chegam ao **Controller**; o Controller atua sobre o **Model** e **seleciona a View**; a View exibe o estado do Model. MVC utiliza o **padrão Observer** para que o Model notifique alterações à View.

Vantagens: (1) **desacoplamento** entre o modelo da aplicação e a camada de exibição; (2) permite a existência de **diferentes camadas de exibição**; (3) **transparência**: o modelo não precisa saber qual camada de exibição é usada.

Modelos Web em Java

Modelo	Quem processa a requisição e define o fluxo	Quem constrói a página	Leitura
Modelo 1	O frontend (JSP, p.ex.)	O mesmo frontend (JSP)	Tudo na página: processamento, fluxo e exibição juntos. Não há Controller separado.
Modelo 2	O backend (servlets, p.ex.)	O frontend (JSP)	É o MVC na Web: servlet = Controller, JSP = View.

Resposta-modelo: exercício 2 da aula 4 (o que é MVC, partes, exemplo, vantagem)

O que é: padrão que divide as responsabilidades da aplicação em três partes. **Model** representa o estado da aplicação (dados); **View** representa a camada de apresentação (exibição); **Controller** representa o fluxo de eventos (ações do usuário), atuando sobre o Model e selecionando a View.

Exemplo: aplicação Web Java no Modelo 2: um servlet (Controller) recebe a requisição "listar produtos", consulta os objetos `Produto` (Model) e encaminha para uma JSP (View) que monta a página. **INFERÊNCIA** No trabalho: componentes `Vue` = View, `@RestController` do Spring = Controller, entidades JPA = Model.

Vantagem: desacoplamento entre modelo e exibição, permitindo várias Views para o mesmo Model (Web, desktop, mobile) sem que o modelo saiba qual está em uso.

Perguntas que o leitor faria

Onde já usei IoC sem saber?

No Swing: você não escreve o laço que espera cliques. Você implementa `jButton1ActionPerformed` e a infraestrutura (a thread de eventos) chama seu código quando oportuno. É exatamente "a infraestrutura provê o laço de eventos; a aplicação apenas trata os eventos".

IoC, Hollywood e DIP são a mesma coisa?

São parentes. **DIP**: dependa de abstrações (estrutura do código). **Hollywood**: o alto nível chama o baixo nível, nunca o contrário (direção das chamadas). **IoC**: o framework controla o fluxo (quem chama quem, em runtime). **DI**: caso particular de IoC para obtenção de dependências.

O crud da aula 6 usa DI?

Não exatamente: a janela ainda **pede** o DAO à fábrica (`getFactory( ... ).createProdutoDAO()`). Isso é Abstract Factory, que já isola a criação do uso. Com DI, a janela **receberia** o `ProdutoDAO` pronto no construtor. No Spring (aula 8, fora da P1) é o container que entrega.

O crud em Swing é MVC?

INFERÊNCIA Parcialmente: `Produto` é o Model e a janela acumula View e Controller (os métodos `ActionPerformed` tratam as ações). A separação em pacotes `view` , `db.vo` , `db.dao` caminha nessa direção.

Por que o Model usa Observer em vez de chamar a View direto?

Se chamasse direto, o Model conheceria a View (acoplamento) e só poderia haver uma. Com Observer, qualquer número de Views se cadastra e é notificado: é a "transparência" listada nas vantagens.

Provável na prova

- Exercícios 2 e 3 da aula 4, praticamente como estão: **definição + exemplo + vantagem**.
- "Diferencie IoC de DI."
- "Reescreva o código abaixo aplicando injeção de dependência" (com o `new Log( ... )` dentro do método).
- "Que padrão GoF o MVC utiliza e para quê?" → Observer, para o Model notificar a View.
- "Diferencie Modelo 1 e Modelo 2."

Módulo 9 · Persistência: DAO, VO e mapeamento objeto-relacional

Por que estudar

É onde a teoria dos módulos anteriores vira estrutura de pacotes. "O que é um DAO? O que é um VO? Como se mapeia um relacionamento 1:N em objetos?" são perguntas diretas, e o exercício da aula 5 é a base do da aula 6.

A motivação: o problema do JDBC na tela

JDBC é o principal mecanismo Java de acesso a bases de dados: acesso por operações básicas a muitos RDBMS, submetendo **sentenças SQL** ou chamadas de **Stored Procedures**. **Problema:** muitas aplicações operam o JDBC na **camada de apresentação (UI)**, o que gera **código redundante** e torna o sistema de **difícil manutenção**. Era exatamente o estado do crud vindo de POO: SQL dentro dos botões da janela.

Os quatro tipos de objeto

Termo	Definição (slide)	No crud
POJO · Plain Old Java Object	Os objetos mais simples em Java: membros privados e getters e setters públicos para todos. Pode ter, além do construtor default, o construtor completo e parciais.	<code>Produto</code> é os três ao mesmo tempo: membros privados + <code>get/set</code> , <code>implements Serializable</code> , e descreve a entidade <code>produto</code> .
JavaBean	Componente Java portável e independente de plataforma , com propriedades customizáveis (ex.: componentes GUI). Normalmente: POJO que implementa <code>Serializable</code> .	
VO · Value Object (ou Transfer Object)	JavaBeans que descrevem as entidades de dados do sistema. São criados pelos DAOs e usados pela aplicação. Permitem o mapeamento objeto-relacional.	
DAO · Data Access Object	Objetos que implementam toda a lógica de acesso ao DB . Fornecem métodos públicos de consulta (ex.: <code>List<Client> getClientsByFirstName(String)</code>). Normalmente implementam CRUD: Create, Retrieve, Update, Delete . Uma Factory + interfaces podem dar acesso a DAOs específicos para RDBMS diferentes.	Interface <code>ProdutoDAO</code> + <code>JDBCProdutoDAO</code> (e depois <code>JPAProdutoDAO</code>).

DAO e VO são **J2EE Design Patterns** (Core J2EE Patterns), não GoF. Se perguntarem "é um dos 23?", a resposta é não.

Mapeamento objeto-relacional (ORM)

Definição: estratégia para **representação de entidades de dados relacionais** (contidas em tabelas de um RDBMS) **como objetos de dados**.

Tipo de objeto	Característica
Objeto persistente	Reflete um estado consistente do sistema: uma alteração em seu estado deve obrigatoriamente gerar alteração de sua entidade no RDBMS.
Objeto de sessão	Transiente: existe durante a execução de uma transação . Ao final, os participantes são comprometidos (commit) e os efeitos são gerados nos objetos persistentes.
Business Objects	Camada adicional de negócio que faz a integração entre os dois tipos acima.

As três regras de mapeamento

No banco	Em Java	Observação do slide
Tabela	Uma classe VO (e uma classe DAO para cada VO)	Uma para cada tabela, com exceção das tabelas de relacionamento .
Coluna (PK ou campo)	Um atributo de classe do VO, com get/set	Tipos do RDBMS representados por equivalentes Java (tabela abaixo).
Relacionamento 1:N (PK do lado 1 exportada como FK no lado N)	Uma coleção na entidade composta (ou agregada)	"As referências das entidades múltiplas são mantidas em coleções."

JDBC type	Java type	JDBC type	Java type
CHAR, VARCHAR, LONGVARCHAR	<code>String</code>	REAL	<code>float</code>
NUMERIC, DECIMAL	<code>java.math.BigDecimal</code>	FLOAT, DOUBLE	<code>double</code>
BIT	<code>boolean</code>	BINARY, VARBINARY, LONGVARBINARY	<code>byte[]</code>
TINYINT	<code>byte</code>	DATE / TIME / TIMESTAMP	<code>java.sql.Date / Time / Timestamp</code>
SMALLINT	<code>short</code>	CLOB / BLOB	<code>Clob / Blob</code>
INTEGER	<code>int</code>	BIGINT	<code>long</code>

Complementos: VOs implementam `get/set` para cada atributo, podem ter construtor completo e, para sistemas distribuídos, devem implementar `Serializable`. DAOs podem definir **interfaces** para desacoplamento em ambientes distribuídos.

Exemplo resolvido 1: tabela produto → VO

`id_produto INTEGER PK AUTO_INCREMENT → Integer idProduto · nome_produto VARCHAR(50) → String nomeProduto · preco_produto VARCHAR(30) → String precoProduto · quantidade_produto INTEGER → Integer quantidadeProduto`. Construtor vazio, construtor completo (4 campos) e parcial sem id (para inserir, já que o id é `AUTO_INCREMENT`). `Get/set` para todos.

Exemplo resolvido 2: o 1:N do slide (setor × empregado)

Banco: `setor(id_setor PK, nome_setor)` e `empregado(id_empregado PK, id_setor FK, nome_empregado)`. A PK do lado 1 (`setor`) foi exportada como FK para o lado N (`empregado`).

Objetos: `class Setor { Integer idSetor; String nomeSetor; List<Empregado> empregados; }` e `class Empregado { Integer idEmpregado; String nomeEmpregado; }`. Repare na **inversão**: no banco quem guarda a ligação é o lado N (a FK); nos objetos quem guarda é o lado 1 (a coleção). O mesmo vale para o exemplo do slide "uma Fatura possui vários Produtos: coleção de objetos Produto é um dos membros de Fatura".

Como fica no código (solução da aula 5)

```
// O CONTRATO: o que a janela enxerga. CRUD = Create, Retrieve, Update, Delete
public interface ProdutoDAO {
    public void create(Produto produto) throws Exception;
    public List<Produto> retrieveAll() throws Exception;
    public Produto retrieveById(int idProduto) throws Exception;
    public void update(Produto produto) throws Exception;
    public void delete(Produto produto) throws Exception;
}

// A IMPLEMENTAÇÃO JDBC: todo o SQL mora aqui, e só aqui
public class JDBCProdutoDAO implements ProdutoDAO {
    public List<Produto> retrieveAll() throws Exception {
        List<Produto> produtos = new ArrayList<>();
        ResultSet result = DBConnection.getInstance().getStatement() // usa o Singleton da aula 2
            .executeQuery("SELECT * FROM produto");
        while (result.next()) { // cada linha vira um VO
            produtos.add(new Produto(result.getInt(1), result.getString(2),
                result.getString(3), result.getInt(4)));
        }
        return produtos; // o DAO CRIA os VOs
    }
    // create/update/delete montam o SQL com os getters do VO e chamam executeUpdate(sql)
}

// NA JANELA (aula 5): nenhum SQL, só VO + DAO
ProdutoDAO produtoDAO = new JDBCProdutoDAO();
produtoDAO.create(new Produto(nome, preco, Integer.parseInt(qtde)));
```

Perguntas que o leitor faria

Por que declarar a variável como ProdutoDAO e não como JDBCProdutoDAO?

Para depender da abstração (DIP). É isso que permite, na aula 6, trocar a implementação por JPA sem alterar as chamadas `create`, `update` ...

Por que tabelas de relacionamento não viram VO?

Uma tabela associativa pura (N:N) só carrega as duas FKs; em objetos esse vínculo já é expresso pelas coleções de cada lado. Criar uma classe para ela seria redundante.

VO é "objeto persistente"?

Conceitualmente ele representa a entidade persistente. Com JDBC puro, porém, a sincronização é manual: alterar o VO não altera o banco até alguém chamar `dao.update(vo)`. Com JPA (módulo 10) o provedor assume essa sincronização dentro da transação.

Por que preço é VARCHAR/String?

Simplificação do exemplo de POO (guarda "30,00" como texto). Pela tabela de tipos, o correto para dinheiro seria `DECIMAL` → `BigDecimal`.

O SQL concatenado com + tem algum problema?

FORA DO SLIDE Sim, é vulnerável a SQL injection; o ideal seria `PreparedStatement`. Não foi tema da aula, mas mostra mais uma vantagem do DAO: a correção seria feita em um único lugar.

Provável na prova

- "Diferencie DAO de VO" / "O que é um POJO? E um JavaBean?"
- "Qual o problema de usar JDBC na camada de apresentação?" → código redundante, difícil manutenção.
- Dado um DER com 1:N, escrever as classes VO (coleção no lado 1).
- "O que é mapeamento objeto-relacional?" + objeto persistente × objeto de sessão.
- "O que significa CRUD?"

Módulo 10 · JPA e Abstract Factory: o crud final

Por que estudar

É a aula mais recente (14/09) e a que junta tudo: Singleton, DAO, VO, ORM automático e Abstract Factory na mesma aplicação. Se cair uma questão "integradora", será sobre este desenho. Atenção: esta aula **não tem slides**, só o roteiro do exercício e a solução.

A evolução do crud: cada aula remove um acoplamento

Versão	Como a janela acessa o banco	Problema que sobra	Padrão / princípio introduzido
POO (ponto de partida)	Abre conexão e executa SQL dentro da própria janela.	JDBC na UI; conexão aberta várias vezes.	—
Aula 2	<code>DBConnection.getInstance().getStatement().executeUpdate(sql)</code>	O SQL ainda está na janela.	Singleton (uma conexão, ponto único de acesso) + parâmetros em <code>crud.properties</code> .
Aula 5	<code>produtoDAO.create(produto)</code>	A janela ainda faz <code>new JDBCProdutoDAO()</code> : conhece a tecnologia.	DAO + VO (Pure fabrication, Indirection, Single-responsibility). Pacotes <code>dao</code> , <code>vo</code> , <code>util</code> , <code>view</code> .
Aula 6	<code>DAOAbstractFactory.getFactory(tipo).createProdutoDAO().create(produto)</code>	—	JPA (ORM automático) + Abstract Factory (Protected variants, Dependency inversion). ComboBox escolhe JDBC ou JPA em runtime.

JPA: o ORM feito por anotações

JPA é a especificação Java de persistência; quem executa é um **provedor**. Na aula 6 usou-se o **EclipseLink (JPA 2.1)** pela biblioteca do NetBeans; o **Hibernate** (apresentado na aula 1 como "framework de persistência que realiza o mapeamento objeto-relacional, com configurações em XML ou anotações, usado de forma transparente pelo JPA") é outro provedor. Por isso o roteiro avisa: "onde vê Hibernate no diagrama, considere JPA".

Anotação	O que declara	Regra de ORM (módulo 9) que ela automatiza
<code>@Entity</code>	A classe é uma entidade persistente.	tabela → classe
<code>@Table(name = "produto")</code>	Nome da tabela correspondente.	tabela → classe
<code>@Id</code>	O atributo é a chave primária.	PK → atributo

<code>@GeneratedValue(strategy = GenerationType.IDENTITY)</code>	O valor é gerado pelo banco (casa com <code>AUTO_INCREMENT</code>).	—
<code>@Column(name = "nome_produto", nullable = false)</code>	Nome da coluna e restrição <code>NOT NULL</code> .	coluna → atributo

persistence.xml (em `META-INF`): define a **Persistence Unit** `crudPU`, com `transaction-type="RESOURCE_LOCAL"`, o `<provider>` (EclipseLink), a lista de `<class>` de entidades (`crud.db.vo.Produto`) e as 4 propriedades de conexão (`javax.persistence.jdbc.url`, `user`, `driver`, `password`). Estratégia de criação de tabelas: **none** (a tabela já existe).

EntityManager × JDBC, operação por operação

CRUD	JDBCProdutoDAO	JPAProdutoDAO	Transação?
Create	<code>executeUpdate("INSERT INTO produto VALUES(null, '...')")</code>	<code>em.persist(produto)</code>	sim
Retrieve (todos)	<code>executeQuery("SELECT * FROM produto")</code> + laço no <code>ResultSet</code>	<code>em.createQuery("SELECT p FROM Produto p", Produto.class).getResultList()</code>	não
Retrieve (id)	<code>"... WHERE id_produto=" + id</code>	<code>"... WHERE p.idProduto=" + id + getSingleResult()</code>	não
Update	<code>executeUpdate("UPDATE produto SET ...")</code>	<code>em.merge(produto)</code>	sim
Delete	<code>executeUpdate("DELETE FROM produto WHERE ...")</code>	<code>em.remove(em.find(Produto.class, id))</code>	sim

Toda escrita fica entre `em.getTransaction().begin()` e `em.getTransaction().commit()`: é o "objeto de sessão" e o *commit* do módulo 9 acontecendo na prática. A consulta usa **JPQL**, que fala de **classes e atributos** (`Produto p`, `p.idProduto`), não de tabelas e colunas (`produto`, `id_produto`).

A Abstract Factory do exercício

Como fica no código (solução da aula 6)

```
public abstract class DAOAbstractFactory {
    public static final int JDBC = 0;           // mesmos valores dos índices da ComboBox
    public static final int JPA = 1;

    public abstract ProdutoDAO createProdutoDAO();    // um create...() por DAO da família

    public static DAOAbstractFactory getFactory(int tipo) {    // escolhe a fábrica concreta
        switch (tipo) {
            case JDBC: return new JDBCDAOFactory();
            case JPA:  return new JPADAOFactory();
            default:   return null;
        }
    }
}

public class JDBCDAOFactory extends DAOAbstractFactory {
    public ProdutoDAO createProdutoDAO() { return new JDBCProdutoDAO(); }
}

public class JPADAOFactory extends DAOAbstractFactory {
    public ProdutoDAO createProdutoDAO() { return JPAProdutoDAO.getInstance(); } // na solução, o DAO JPA é
Singleton
}

// NA JANELA: de new JDBCProdutoDAO() para
ProdutoDAO produtoDAO =
    DAOAbstractFactory.getFactory(jComboBox1.getSelectedIndex()).createProdutoDAO();
```

Papel no padrão	Classe no crud
AbstractFactory	DAOAbstractFactory
ConcreteFactory	JDBCDAOFactory , JPADAOFactory
AbstractProduct	ProdutoDAO (interface)
ConcreteProduct	JDBCProdutoDAO , JPAProdutoDAO
Client	CrudProduto (janela)

Exemplo resolvido: o que acontece quando o usuário escolhe "JPA" e clica em salvar

1) `jComboBox1.getSelectedIndex()` devolve **1** (ordem das opções: JDBC, JPA). 2) `getFactory(1)` cai em `case JPA` e devolve uma `JPADAOFactory`. 3) `createProdutoDAO()` devolve a instância única de `JPAProdutoDAO`, cujo construtor criou `EntityManagerFactory` ("crudPU") e `EntityManager`. 4) `create(produto)` faz `begin` → `persist` → `commit`. 5) O provedor lê as anotações do VO e gera o `INSERT INTO produto...` sozinho. A janela não sabe qual tecnologia foi usada.

Exemplo resolvido: quais padrões e princípios aparecem no crud final?

Singleton: `DBConnection` e, na solução, `JPAProdutoDAO`. **Abstract Factory:** `DAOAbstractFactory` e as duas fábricas. **DAO e VO (J2EE).** **Observer:** os Listeners dos botões Swing. Princípios: **Pure fabrication, Indirection, Protected variants, Polymorphism, Dependency inversion, Open-closed.**

Perguntas que o leitor faria

Se só existe um DAO (produto), por que Abstract Factory e não Factory Method?

Porque o desenho prevê a **família**: quando o sistema tiver `ClienteDAO` e `FaturaDAO`, a fábrica abstrata ganha `createClienteDAO()` e `createFaturaDAO()`, e cada fábrica concreta entrega a família inteira na mesma tecnologia. É o que o slide chama de "criar famílias de objetos relacionados por meio de uma única interface".

O switch dentro de getFactory não fere o Open-closed?

Em parte, sim: uma terceira tecnologia exige mexer nesse `switch`. Mas a modificação fica **confinada a um ponto**; a janela e os DAOs existentes continuam fechados. É a "interface estável em volta do ponto de variação" de Protected variants.

Por que o delete do JPA faz em .find() antes de em.remove()?

O `remove` só aceita uma entidade **gerenciada** pelo EntityManager. O `produto` que veio da janela é um objeto comum; `find` busca a versão gerenciada pelo id, e é essa que se remove.

Por que as consultas não abrem transação?

Porque não alteram o estado persistente. Transação (begin/commit) é necessária para `persist`, `merge` e `remove`.

O roteiro e a solução diferem em algo?

Sim: no roteiro, `JPAProdutoDAO` tem construtor público; na solução do professor ele virou **Singleton** (construtor privado + `getInstance()`), para não recriar o `EntityManagerFactory` a cada operação, e ganhou `finalize()` fechando `em` e `emf`.

Provável na prova

- "Explique o padrão Abstract Factory e dê um exemplo" → use o crud: fábrica abstrata, duas concretas, produto abstrato `ProdutoDAO`.
- "Que vantagem a fábrica trouxe para a janela?" → isolar a criação do uso; trocar JDBC por JPA em runtime sem alterar as chamadas.
- "Para que servem `@Entity`, `@Id`, `@Column`?"
- "Identifique os padrões presentes no diagrama/código" (questão integradora).

Java: colinha de código

Primeiro, "trecho → o que ele responde", agrupado por projeto. Depois, os scripts completos para treinar escrita à mão e a lista de erros frequentes.

Projeto designpatterns (aulas 2 e 3): a assinatura de cada padrão

Se você vir isto...	...é
<code>private static X instance + construtor private + static getInstance()</code>	Singleton
método que recebe parâmetro e faz <code>return new SubclasseA()</code> ou <code>new SubclasseB()</code>	Factory Method
classe abstrata com <code>abstract createX()</code> + <code>static getFactory(tipo)</code>	Abstract Factory
<code>Director(builder)</code> chamando <code>buildA(); buildB(); buildC();</code> e <code>getProduct()</code>	Builder
<code>implements Cloneable + clone()</code>	Prototype
classe que implementa a interface esperada e, dentro, converte e chama outra classe incompatível	Adapter
classe abstrata com atributo <code>protected Implementor impl</code> recebido no construtor	Bridge
classe que <code>implements Shape</code> e contém <code>List<Shape></code>	Composite
<code>new A(new B(new C()))</code> , todos da mesma interface, cada um chamando <code>super.metodo()</code> e acrescentando algo	Decorator
classe com vários subsistemas como atributos e um método que chama todos	Façade
fábrica com <code>HashMap</code> estático: "se já existe, devolve o mesmo"	Flyweight
mesma interface do real + <code>if (permitido) real.metodo()</code>	Proxy
<code>setNextChain(next)</code> + "se não trato, <code>chain.dispense(...)</code> "	Chain of Responsibility
interface com <code>execute()</code> + invocador com <code>setCommand()</code>	Command
<code>hasNext()</code> / <code>next()</code>	Iterator
colegas guardam referência a um objeto central e chamam <code>mediator.sendMessage(msg, this)</code>	Mediator
<code>save()</code> devolve objeto de estado + <code>undo / restore</code>	Memento
lista de listeners + <code>addListener()</code> + laço chamando o método de callback	Observer
<code>context.setState(s); context.doAction()</code>	State
<code>objeto.fazer(new AlgoritmoX())</code> com interface comum	Strategy
<code>public final void algoritmo()</code> chamando métodos <code>abstract</code>	Template Method
<code>element.accept(visitor)</code> → <code>visitor.visit(this)</code>	Visitor

Projeto crud (aulas 2, 5 e 6)

Trecho	O que responde
<code>ResourceBundle.getBundle("crud/crud").getString("jdbc.url")</code>	Lê os parâmetros de conexão do <code>crud.properties</code> (o "recurso de internacionalização" do item d da aula 2).

<code>Class.forName(driver) · DriverManager.getConnection(url, user, pwd) · connection.createStatement()</code>	Carrega o driver, abre a conexão e cria o Statement. Feito uma vez , no construtor privado do Singleton.
<code>statement.executeUpdate(sql)</code>	INSERT, UPDATE, DELETE.
<code>statement.executeQuery(sql) → ResultSet · result.next() · result.getInt(1)</code>	SELECT; percorre as linhas; colunas contam a partir de 1 .
<code>Persistence.createEntityManagerFactory("crudPU") · emf.createEntityManager()</code>	Sobe o JPA lendo a Persistence Unit do <code>persistence.xml</code> .
<code>em.getTransaction().begin() ... commit()</code>	Delimita a transação das escritas.
<code>em.persist(o) / em.merge(o) / em.remove(em.find(C.class, id))</code>	Create / Update / Delete.
<code>em.createQuery("SELECT p FROM Produto p", Produto.class).getResultList()</code>	Retrieve com JPQL.

Scripts completos

1 · Singleton de conexão (DBConnection, aula 2, condensado)

```
public class DBConnection {
    private static DBConnection instance = null;
    private Connection connection;
    private Statement statement;

    private DBConnection() throws Exception { // construtor privado abre a conexão
        ResourceBundle rb = ResourceBundle.getBundle("crud/crud");
        Class.forName(rb.getString("jdbc.driver"));
        connection = DriverManager.getConnection(rb.getString("jdbc.url"),
            rb.getString("jdbc.user"), rb.getString("jdbc.password"));
        statement = connection.createStatement();
    }

    public static DBConnection getInstance() throws Exception {
        if (instance == null) { instance = new DBConnection(); }
        return instance;
    }

    public Statement getStatement() { return statement; } // item e) do exercício

    @Override
    protected void finalize() throws Throwable { // item f): "destrutor"
        statement.close();
        connection.close();
    }
}

// crud.properties
// jdbc.driver=com.mysql.cj.jdbc.Driver
// jdbc.url=jdbc:mysql://localhost:3306/curso
// jdbc.user=aluno
// jdbc.password=segredo
```

2 · VO com anotações JPA (aulas 5 e 6)

```
@Entity
@Table(name = "produto")
public class Produto implements Serializable {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_produto")
    private Integer idProduto;
    @Column(name = "nome_produto", nullable = false)
    private String nomeProduto;
    @Column(name = "preco_produto", nullable = false)
    private String precoProduto;
    @Column(name = "quantidade_produto", nullable = false)
    private Integer quantidadeProduto;

    public Produto() { } // default (JPA exige)
    public Produto(String nome, String preco, int qtde) { ... } // parcial, para inserir
    public Produto(int id, String nome, String preco, int qtde) { ... } // completo
    // getters e setters de todos os atributos
}
```

3 · DAO com JPA (aula 6)

```
public class JPAProdutoDAO implements ProdutoDAO {
    private EntityManagerFactory emf;
    private EntityManager em;
    public JPAProdutoDAO() {
        emf = Persistence.createEntityManagerFactory("crudPU");
        em = emf.createEntityManager();
    }
    public void create(Produto produto) throws Exception {
        em.getTransaction().begin();
        em.persist(produto);
        em.getTransaction().commit();
    }
    public List<Produto> retrieveAll() throws Exception {
        return em.createQuery("SELECT p FROM Produto p", Produto.class).getResultList();
    }
    public Produto retrieveById(int idProduto) throws Exception {
        return em.createQuery("SELECT p FROM Produto p WHERE p.idProduto=" + idProduto,
            Produto.class).getSingleResult();
    }
    public void update(Produto produto) throws Exception {
        em.getTransaction().begin();
        em.merge(produto);
        em.getTransaction().commit();
    }
    public void delete(Produto produto) throws Exception {
        em.getTransaction().begin();
        em.remove(em.find(Produto.class, produto.getIdProduto()));
        em.getTransaction().commit();
    }
}
```

4 · Template Method, Strategy e Command em miniatura (para comparar)

```
// TEMPLATE METHOD: herança, esqueleto final, passos abstratos
public abstract class HouseTemplate {
    public final void buildHouse() { buildFoundation(); buildPillars(); buildWalls(); buildWindows(); }
    private void buildFoundation() { ... } // fixo
    public abstract void buildPillars(); // varia na subclasse
    public abstract void buildWalls();
    private void buildWindows() { ... }
}

// STRATEGY: composição, algoritmo inteiro passado como objeto
public interface PaymentStrategy { void pay(int amount); }
public void pay(PaymentStrategy paymentMethod) { paymentMethod.pay(calculateTotal()); }
cart.pay(new PaypalStrategy("myemail@example.com", "mypwd")); // itens 10 + 40 → "50 paid using Paypal."

// COMMAND: a ação vira objeto; o invocador não conhece o receptor
public interface Command { void execute(); }
public class LightOnCommand implements Command {
    Light light;
    public LightOnCommand(Light light) { this.light = light; }
    public void execute() { light.switchOn(); }
}
control.setCommand(new LightOnCommand(light));
control.pressButton(); // RemoteControl só chama command.execute()
```

Erros frequentes

Erro	Sintoma	Correção
Construtor do Singleton <code>public</code>	Qualquer um faz <code>new</code> : deixa de ser instância única.	<code>private</code> .
<code>getInstance()</code> ou <code>instance</code> sem <code>static</code>	Não compila / não há como chamar sem objeto.	Ambos <code>static</code> .
<code>void</code> no "construtor" (como no slide de DI)	Vira método comum; atributo injetado fica <code>null</code> → <code>NullPointerException</code> .	Construtor não tem tipo de retorno.
Escrita JPA sem <code>begin()</code> / <code>commit()</code>	Nada é gravado, ou exceção de transação.	Envolver <code>persist</code> , <code>merge</code> , <code>remove</code> .
JSQL com nome de tabela/coluna (<code>FROM produto</code> , <code>id_produto</code>)	Erro de sintaxe da consulta.	Usar classe e atributo: <code>FROM Produto p</code> , <code>p.idProduto</code> .
<code>em.remove(produto)</code> direto	<code>IllegalArgumentException</code> : entidade não gerenciada.	<code>em.remove(em.find(Produto.class, id))</code> .
Esquecer <code><class></code> no <code>persistence.xml</code> ou nome errado da PU	"Produto is not a known entity type" / "No Persistence provider for EntityManager named..."	"Include Entity Classes" e conferir <code>"crudPU"</code> .
<code>result.getInt(0)</code>	<code>SQLException</code> : índice inválido.	Colunas do <code>ResultSet</code> começam em 1.
Driver MySQL fora das bibliotecas	<code>ClassNotFoundException</code> : <code>com.mysql.cj.jdbc.Driver</code> .	Adicionar o conector à Library do projeto.
Ordem da ComboBox diferente das constantes	Escolhe "JPA" e roda JDBC.	Opções na ordem JDBC (0), JPA (1).

Listas que caem (para decorar)

Esta disciplina não tem fórmulas: o equivalente são as **listas fechadas** que o professor mostrou. Aqui estão todas em uma página, só as que aparecem nos slides, com um exercício de classificação resolvido no final.

Arquitetura (aula 1)

Arquitetura = {Elementos, Organização, Decisões}

Escalas de padrão (5): arquitetura · análise · projeto · teste e idiomas · implementação

Elementos de um padrão GoF (4): nome · problema · solução · consequências

Exemplos de estilos (8): cliente-servidor · em camadas · baseado em componentes · baseado em regras · centrado em dados · orientado a eventos · orientado a serviços · objetos distribuídos

Elementos estáticos (3): software · dados · hardware

Blackboard (3): blackboard · base de conhecimento · controle

Middleware cliente-servidor (3): procedimentos remotos · objetos remotos · serviços distribuídos

Uso de framework (3): conjunto de ferramentas · denominador comum de arquiteturas · sistema que possui controle

GRASP (9) · Larman, 1997

Information expert · Creator · Controller · Indirection · Low coupling · High cohesion · Polymorphism · Protected variants · Pure fabrication

Creator (5 condições): B contém A · agrega A · tem dados de inicialização · registra A · usa intensamente A

Acoplamento (4 formas): atributo · obter instância · estender · implementar interface

Indirection → Adapter, Façade, Observer

Exemplo resolvido: classifique em 30 segundos

Lista: *Proxy, Memento, Builder, Bridge, Strategy, Prototype, Visitor, Flyweight.*

GoF (aulas 2 e 3) · 23 = 5 + 7 + 11

Criação (5): Singleton · Factory Method · Abstract Factory · Builder · Prototype

Estruturais (7): Adapter · Bridge · Composite · Decorator · Façade · Flyweight · Proxy

Comportamentais (11): Chain of Responsibility · Command · Interpreter · Iterator · Mediator · Memento · Observer · State · Strategy · Template Method · Visitor

Builder (4): Builder · Concrete Builder · Director · Product

Autores (4): Gamma · Helm · Johnson · Vlissides · livro de 1995

Truque: os estruturais estão em ordem alfabética (A, B, C, D, F, F, P) e os comportamentais também (C, C, I, I, M, M, O, S, S, T, V).

Decore os 5 de criação e os 7 estruturais; o que sobrar é comportamental.

SOLID (5) · Martin, 2000

Single-responsibility · **Open-closed** · **Liskov substitution** · **Interface segregation** · **Dependency inversion**

Liskov (contrato): pré-condições não reforçadas · pós-condições não enfraquecidas · invariantes preservados · encapsulamento mantido

MVC e persistência

MVC: Model = estado (dados) · View = apresentação (exibição) · Controller = fluxo de eventos (ações)

CRUD: Create · Retrieve · Update · Delete

ORM: tabela → VO (+ DAO) · coluna → atributo · 1:N → coleção no lado 1

Criação: **Builder, Prototype** (estão entre os 5). Estruturais: **Proxy, Bridge, Flyweight** (estão entre os 7). O resto é comportamental: **Memento, Strategy, Visitor**. Teste de sanidade pelo significado: criação "fabrica" objetos; estrutural "monta/embrulha" objetos; comportamental descreve "quem faz o quê e quando".

Auto-teste · flashcards

Tente responder em voz alta antes de conferir o gabarito, que começa na página seguinte às perguntas. Se errar, volte ao módulo indicado entre colchetes.

- | | |
|---|--|
| <div>1 Qual a definição de arquitetura de Perry e Wolf?</div> <div>2 O que é um padrão?</div> <div>3 Padrão de arquitetura × estilo de arquitetura</div> <div>4 Os 4 elementos essenciais de um padrão de projeto</div> <div>5 Framework horizontal × vertical</div> <div>6 Elementos estáticos × dinâmicos</div> <div>7 Como os componentes se comunicam no estilo repositório?</div> <div>8 O que o blackboard tem que o repositório não tem?</div> <div>9 Série de transformações em que a saída de um módulo alimenta o seguinte. Qual estilo?</div> <div>10 Cliente-servidor exige computadores diferentes?</div> <div>11 Quantos são os padrões GoF e como se dividem?</div> <div>12 Os três ingredientes de um Singleton em Java</div> <div>13 getInstance("First") e depois getInstance("Second"): o que getMessage() devolve na segunda?</div> <div>14 Factory Method × Abstract Factory</div> <div>15 Os 4 componentes do Builder</div> <div>16 Adapter × Façade</div> <div>17 Proxy × Decorator</div> <div>18 Qual padrão representa hierarquias em árvore tratando folha e composto igualmente?</div> | <div>19 O que a Bridge desacopla, e como?</div> <div>20 Observer: quem são Sujeito e Observer no exercício da aula 3?</div> <div>21 Por que o método template costuma ser final?</div> <div>22 Strategy × State</div> <div>23 Qual a limitação do Visitor?</div> <div>24 Quais alternativas têm só padrões comportamentais? a) Command, Observer, Iterator · b) Singleton, Visitor, State</div> <div>25 Classe enorme com funcionalidades não relacionadas: qual GRASP foi ferido?</div> <div>26 Quais padrões GoF são especializações de Indirection?</div> <div>27 O que é Pure fabrication? Dê o exemplo do slide.</div> <div>28 Enuncie o Dependency inversion principle</div> <div>29 As restrições de contrato de Liskov</div> <div>30 IoC × DI</div> <div>31 As três partes do MVC e o padrão GoF que ele usa</div> <div>32 Modelo 1 × Modelo 2 (Web em Java)</div> <div>33 DAO × VO</div> <div>34 Como um 1:N do banco aparece nos objetos?</div> <div>35 Para que servem @Entity, @Id e @Column?</div> <div>36 O que faz DAOAbstractFactory.getFactory(1).createProdutoDAO()?</div> |
|---|--|

Gabarito do auto-teste

1 Qual a definição de arquitetura de Perry e Wolf?

Arquitetura = {Elementos, Organização, Decisões}: elementos arquiteturais (de dados, de processamento, de conexão) com alguma organização, definidos por decisões tomadas para satisfazer objetivos e restrições. [M1]

2 O que é um padrão?

Template (receita) de solução para um problema recorrente, comprovadamente útil em um determinado contexto. É instanciado pela vinculação de valores a seus parâmetros. [M1]

3 Padrão de arquitetura × estilo de arquitetura

Padrão: um template pronto para um problema arquitetural recorrente. Estilo: delimita o conjunto de templates (e de componentes/conectores) que podem ser escolhidos e impõe uniformidade à arquitetura. [M1]

4 Os 4 elementos essenciais de um padrão de projeto

Nome, problema (contexto), solução (elementos e relacionamentos) e consequências (vantagens e desvantagens). [M1]

5 Framework horizontal × vertical

Horizontal: generalista, flexível, funcionalidades que não são de negócio (toolkits gráficos). Vertical: domínio específico, alinhado ao negócio (aplicações estatísticas ou financeiras). [M1]

6 Elementos estáticos × dinâmicos

Estáticos: as partes (software, dados, hardware) e como se relacionam. Dinâmicos: o comportamento em execução e como o sistema reage a estímulos internos e externos. [M1]

7 Como os componentes se comunicam no estilo repositório?

Indiretamente, pelos dados centralizados: cada cliente opera independente, lendo ou modificando o repositório. [M2]

8 O que o blackboard tem que o repositório não tem?

Um componente de controle que monitora o estado do blackboard e decide qual componente da base de conhecimento executa; a solução é incremental. [M2]

9 Série de transformações em que a saída de um módulo alimenta o seguinte. Qual estilo?

Duto e filtro (pipe and filter); em linha única chama-se pipeline ou sequencial por lotes. Filtros são independentes e desconhecem os vizinhos. [M2]

10 Cliente-servidor exige computadores diferentes?

Não: "pode ser implementada em um computador único". Geralmente é integrada por middleware (RPC, objetos remotos, serviços distribuídos). [M2]

11 Quantos são os padrões GoF e como se dividem?

23: 5 de criação, 7 estruturais, 11 comportamentais. Livro de 1995, de Gamma, Helm, Johnson e Vlissides. [M3]

12 Os três ingredientes de um Singleton em Java

Atributo `private static` da própria classe; construtor `private`; método `public static getInstance()` que cria só se for `null`. [M3]

13 `getInstance("First")` e depois `getInstance("Second")`: o que `getMessage()` devolve na segunda?

"First". A instância já existe, o construtor não roda de novo e o novo parâmetro é ignorado. [M3]

14 Factory Method × Abstract Factory

Factory Method: um método decide qual subclasse de um produto instanciar. Abstract Factory: uma interface de fábrica com várias ConcreteFactories, cada uma criando uma família de objetos relacionados; isola a criação do uso. [M3]

15 Os 4 componentes do Builder

Builder (interface dos passos), Concrete Builder (monta as partes), Director (controla o algoritmo de construção), Product (o objeto complexo). [M3]

16 Adapter × Façade

Adapter converte uma interface incompatível (código de terceiros) na que o cliente espera. Façade oferece uma interface nova, simplificada e de alto nível para um subsistema complexo. [M4]

17 Proxy × Decorator

Ambos implementam a interface do objeto embrulhado. Proxy controla o acesso (intercepta, checa permissão, delega). Decorator acrescenta funcionalidade em tempo de execução, sem afetar outras instâncias. [M4]

18 Qual padrão representa hierarquias em árvore tratando folha e composto igualmente?

Composite. Usado quando há definição recursiva: composites contêm componentes que podem ser composites. [M4]

19 O que a Bridge desacopla, e como?

Desacopla uma abstração de sua implementação, por composição: a abstração mantém uma instância do Implementor (interface) e direciona as requisições ao ConcretImplementor. Ex.: Shape contém Color. [M4]

20 Observer: quem são Sujeito e Observer no exercício da aula 3?

Sujeito: `Origem` (mantém a lista, `addMessageListener`, itera e chama `messageArrived`). Observers: instâncias de `Ouvinte`, que implementam `MessageEventListener`. O evento é `MessageEvent` `extends` `EventObject`. [M5]

21 Por que o método `template` costuma ser final?

Para evitar que subclasses o reimplamentem e mudem o algoritmo base; elas só implementam os passos específicos. [M5]

22 Strategy × State

Mesma estrutura. Strategy: algoritmos alternáveis escolhidos pelo cliente. State: o comportamento muda conforme o estado atual do objeto, guardado em um Context. [M5]

23 Qual a limitação do Visitor?

Adicionar novas classes à coleção normalmente obriga a modificar todas as classes Visitor. Em troca, permite criar novas operações sem alterar as classes visitadas. [M5]

24 Quais alternativas têm só padrões comportamentais? a) Command, Observer, Iterator · b) Singleton, Visitor, State

Só a). Em b), Singleton é de criação. [M5]

25 Classe enorme com funcionalidades não relacionadas: qual GRASP foi ferido?

High cohesion: as responsabilidades do elemento devem ser relacionadas e focadas (poucas tarefas, poucos métodos, altamente relacionados). [M6]

26 Quais padrões GoF são especializações de Indirection?

Adapter, Façade e Observer. [M6]

27 O que é Pure fabrication? Dê o exemplo do slide.

Atribuir responsabilidades acessórias (não do domínio) a classes artificiais, preservando alta coesão, baixo acoplamento e reuso. Exemplo: classes que realizam persistência em um BD. [M6]

28 Enuncie o Dependency inversion principle

Módulos de alto nível não devem depender de módulos de baixo nível; ambos devem depender de abstrações. Abstrações não dependem de detalhes; detalhes dependem de abstrações. Permite acoplamento fraco. [M7]

29 As restrições de contrato de Liskov

No subtipo: pré-condições não podem ser reforçadas, pós-condições não podem ser enfraquecidas, invariantes e encapsulamento devem ser preservados. [M7]

30 IoC × DI

IoC: inverte o fluxo de controle; o container chama o código da aplicação quando oportuno. DI: a estrutura passa a dependência (serviço) ao cliente, em vez de o cliente criá-la. DI é IoC aplicada à obtenção de dependências. [M8]

31 As três partes do MVC e o padrão GoF que ele usa

Model = estado/dados; View = apresentação; Controller = fluxo de eventos/ações. Usa Observer para o Model notificar alterações à View. Vantagem: desacopla modelo e exibição, permitindo várias Views. [M8]

32 Modelo 1 × Modelo 2 (Web em Java)

Modelo 1: o frontend (JSP) processa a requisição, define o fluxo e constrói a página. Modelo 2: o backend (servlet) processa e define o fluxo; o frontend (JSP) só constrói a página. [M8]

33 DAO × VO

DAO: implementa toda a lógica de acesso ao BD (CRUD) e cria os VOs. VO (Transfer Object): JavaBean que descreve uma entidade de dados, com atributos privados, get/set e Serializable. [M9]

34 Como um 1:N do banco aparece nos objetos?

No banco, a PK do lado 1 vai como FK para o lado N. Nos objetos, a entidade do lado 1 mantém uma coleção das entidades do lado N (Setor tem List<Empregado>). [M9]

35 Para que servem @Entity, @Id e @Column?

@Entity marca a classe como entidade persistente; @Id indica a chave primária; @Column liga o atributo à coluna (nome, nullable). Automatizam o mapeamento objeto-relacional. [M10]

36 O que faz DAOAbstractFactory.getFactory(1).createProdutoDAO()?

getFactory(1) devolve a JPADAOFactory (JPA = 1); createProdutoDAO() devolve um JPAProdutoDAO, visto pela janela apenas como ProdutoDAO. [M10]

Lacunas do material e o que NÃO cai

Recorte usado neste guia

Classe	Temas	Evidência
Núcleo	23 padrões GoF (módulos 3 a 5) · GRASP, SOLID, IoC, DI, MVC (6 a 8)	Slide + código ou exercício + sua anotação ("cai na prova", "conteúdo pesado").
Núcleo prático	Singleton, DAO/VO, Abstract Factory + JPA no crud (3, 9, 10)	Slide + exercício com solução. Sem anotação sua para as aulas 5 e 6.
Conceitual	Definições de arquitetura, estilos (1 e 2)	Slide + 2 exercícios dissertativos + anotação.
Lacuna	Teoria de JPA/Hibernate	A aula 6 não tem slides; o módulo 10 foi construído a partir do roteiro e do código da solução.
Fora	Aulas 7 a 10 e o trabalho	Datas posteriores à P1 (28/09 em diante).

O que eu não tive e você deve conferir

- **Plano de ensino / ementa oficial:** não está na pasta. Só há a linha da sua anotação ("Arquitetura de softwares orientados a objetos. Padrões de projetos"). O recorte foi feito pelas datas: P1 em 21/09, última aula dada em 14/09 (aula 6).
- **Formato da prova:** não consta em nenhum material. Os exercícios sugerem dissertativas curtas (definição + exemplo + vantagem), cenário → padrão/princípio, e múltipla escolha estilo concurso. Trecho de código para escrever ou completar é possível, principalmente Singleton e DI. **INFERÊNCIA**
- **Slides que são só figura:** os diagramas UML dos padrões (aulas 2 e 3), "Framework × padrão × arquitetura", "Decisões arquiteturais", os exemplos de repositório, duto e filtro, camadas e cliente-servidor (aula 1), o diagrama do MVC e o DAOFactory (aulas 4 e 5) não têm texto extraível e não puderam ser lidos nesta máquina. Compensei os de GoF com o código-fonte do projeto `designpatterns`, lido por inteiro. **Abra os PDFs e olhe essas figuras pelo menos uma vez**, em especial os exemplos da aula 1.
- **Anotações das aulas 5 e 6:** não há. Se o professor deu alguma ênfase oral nessas aulas, ela não está refletida aqui.
- **Anotação da aula 3:** a tabela está cortada na exportação (aparecem só Chain of Responsibility, Mediator, Strategy e o início de "Command" e "Observer"). Tratei esses cinco como os destacados pelo professor, junto com Singleton, Builder, Bridge e Façade da aula 2.

Ferramentas citadas na apresentação (baixa probabilidade, uma linha cada)

Ferramenta	Definição do slide
Hibernate (com JPA)	Framework de persistência de dados; realiza o mapeamento objeto-relacional; configurações em XML ou anotações; usado de forma transparente pelo JPA.
Spring	Framework Java que permite inversão de controle ; realiza a injeção de dependências ; muito usado para Web (MVC, Rest-WS).
Spring Boot	Framework para criação de projetos Spring: aplicações autônomas com Tomcat embarcado, configuração automática, POMs para Maven e Gradle.
Maven	Automatiza a construção de projetos; obtém dependências automaticamente; configuração em arquivo XML (POM).
Vue.js	Framework JavaScript progressivo para interfaces de usuário (view layer); suporta Single-Page Applications.

Fora da P1 (servem para o trabalho, entregas em 16 e 23/11)

- Aula 7 (28/09): arquiteturas e sistemas Web, programação para Web, tecnologias client side.
- Aula 8 (05 e 19/10): framework backend Spring e Spring Boot.
- Aula 9 (26/10): Vue.js.
- Aula 10 (09/11): integração do frontend Vue.js com backend Spring Boot.
- Trabalho (30% da nota): aplicação Web frontend + backend com microserviços REST, Spring Boot, JPA/Hibernate sobre MySQL/MariaDB, login e senha, pelo menos duas tabelas com relacionamento 1:N. As duas tarefas valem 10% cada.

Sugestão: se conseguir falar com o professor ou com colegas antes da prova, confirme o formato (tem código? pode consultar?) e se a aula 6 entra.